

Operating System Concepts

A. Silberschatz, P. B. Galvin, and G. Gagne, 10th edition

Chapter 6: Synchronization Tools

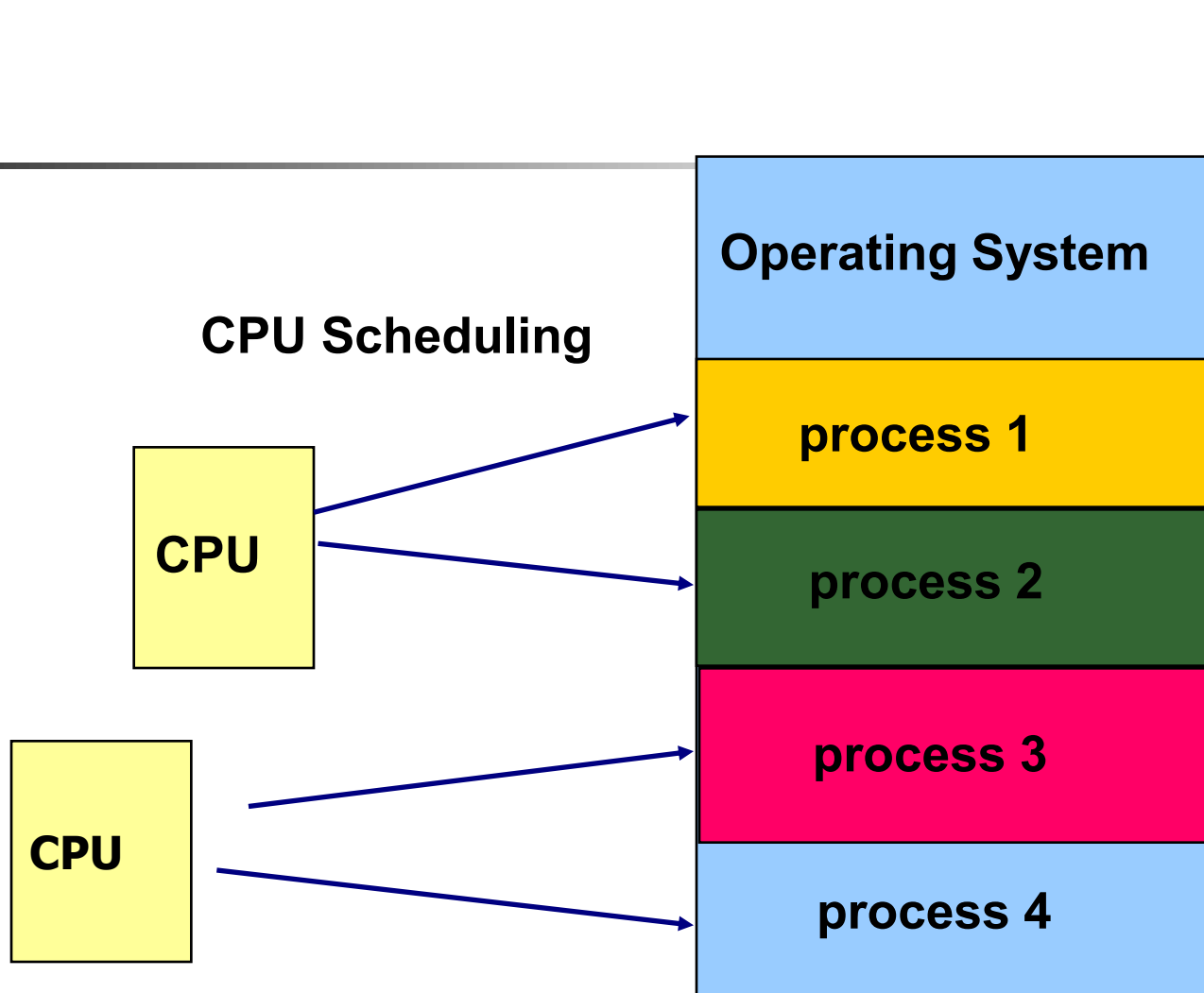
Mei-Ling Chiang, Professor

Dept. of Information Management

National Chi-Nan University, Taiwan, ROC



2025/12/11



Concurrent Execution

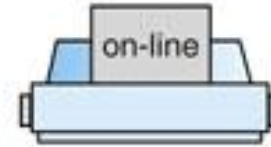
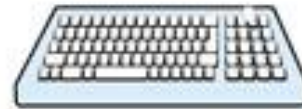
Application programs

disks

mouse

keyboard

printer



CPU

disk
controller

USB controller

graphics
adapter

OS

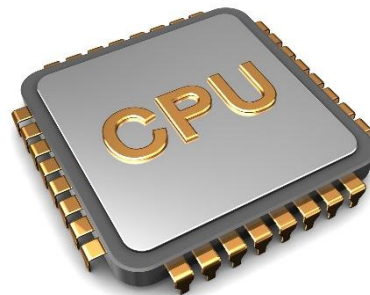
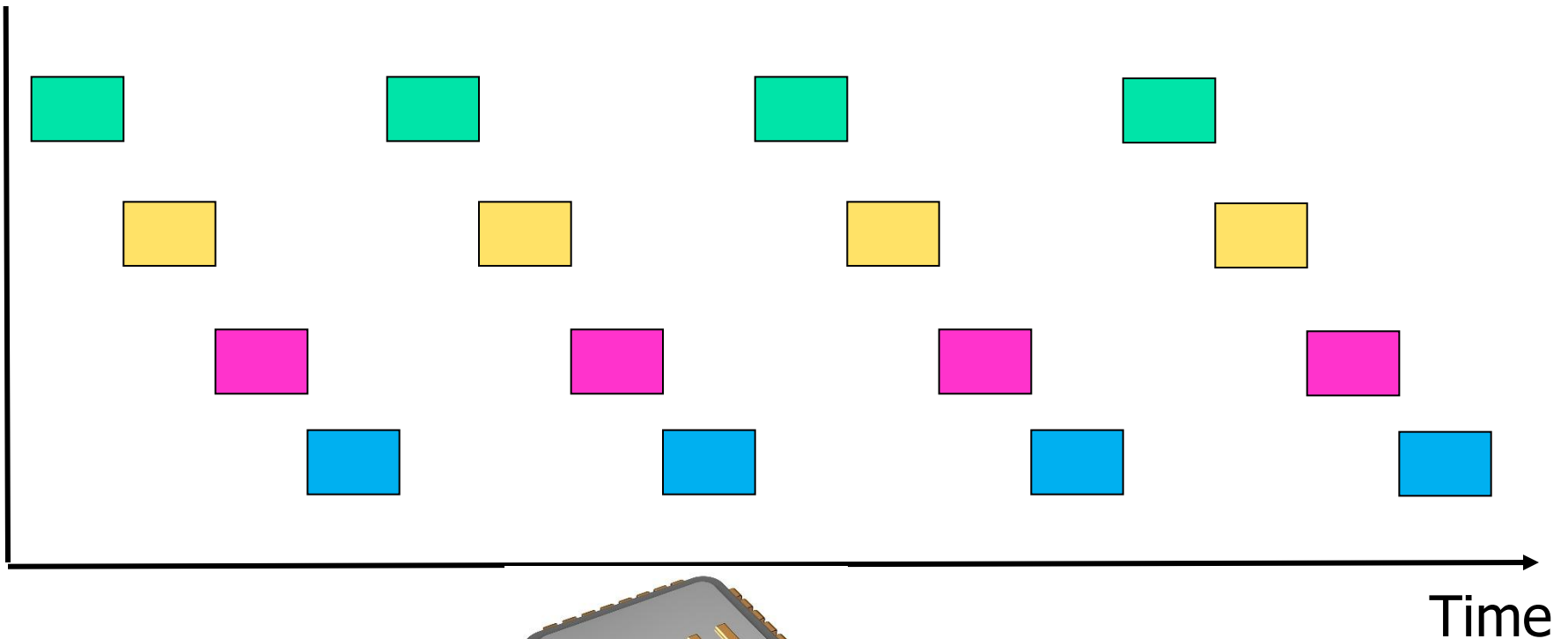
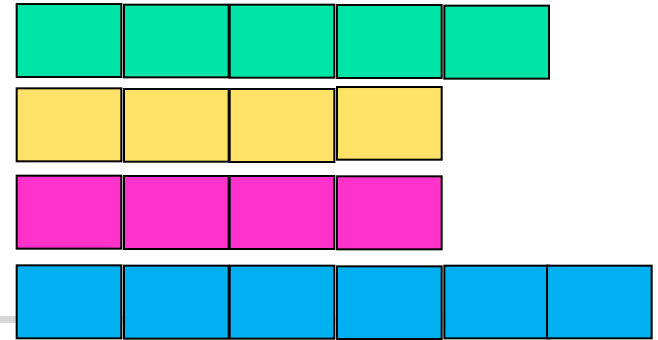
Process
#1

Process
#2

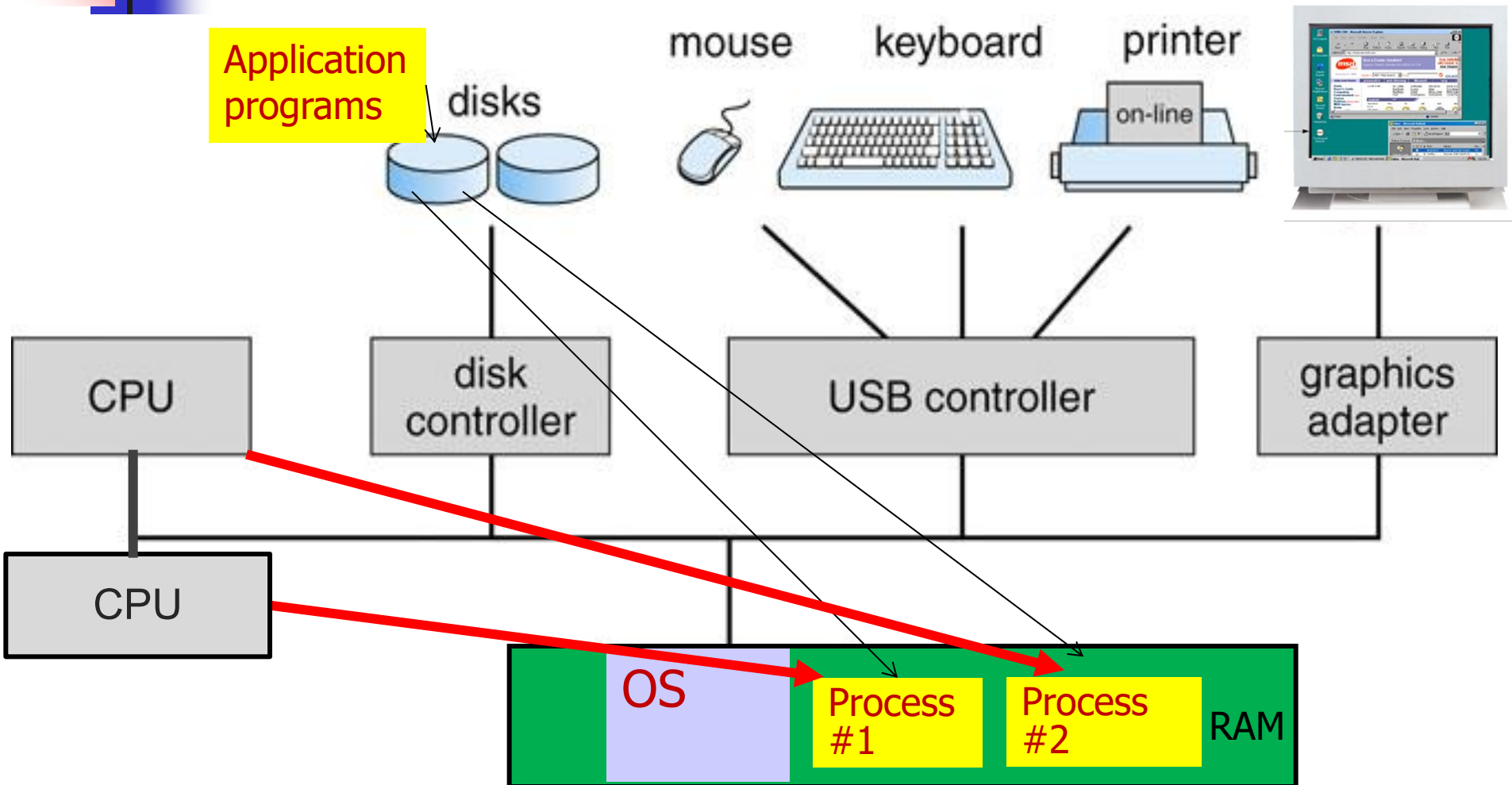
RAM

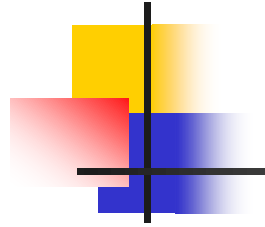
Time-sharing systems

Tasks



Parallel Execution

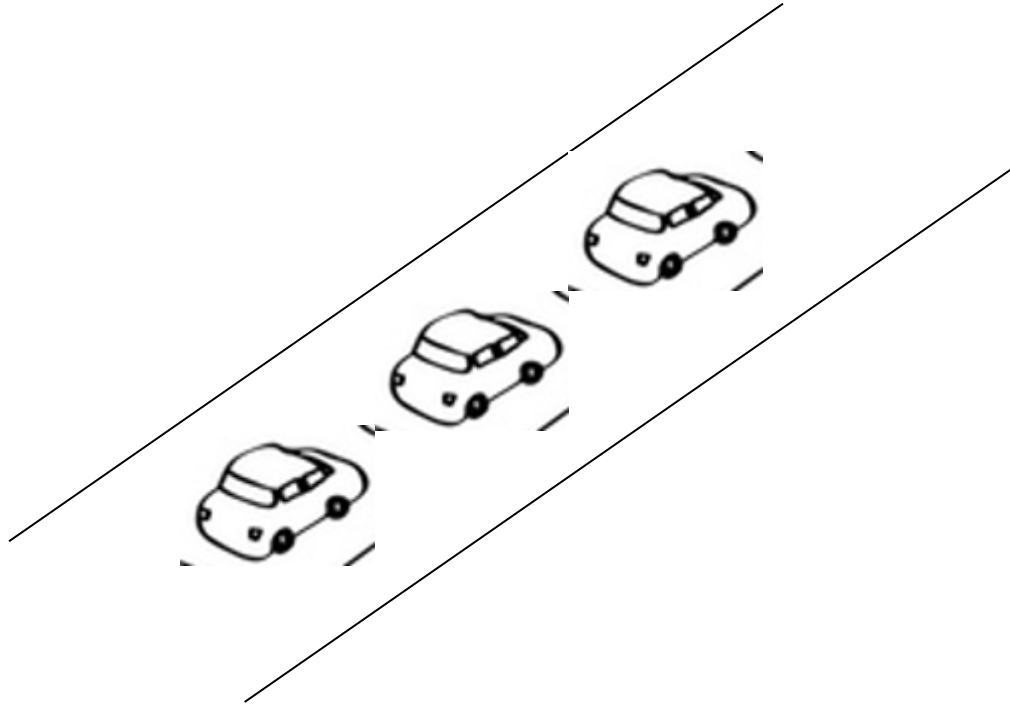


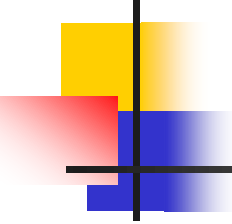


Independent Process ?

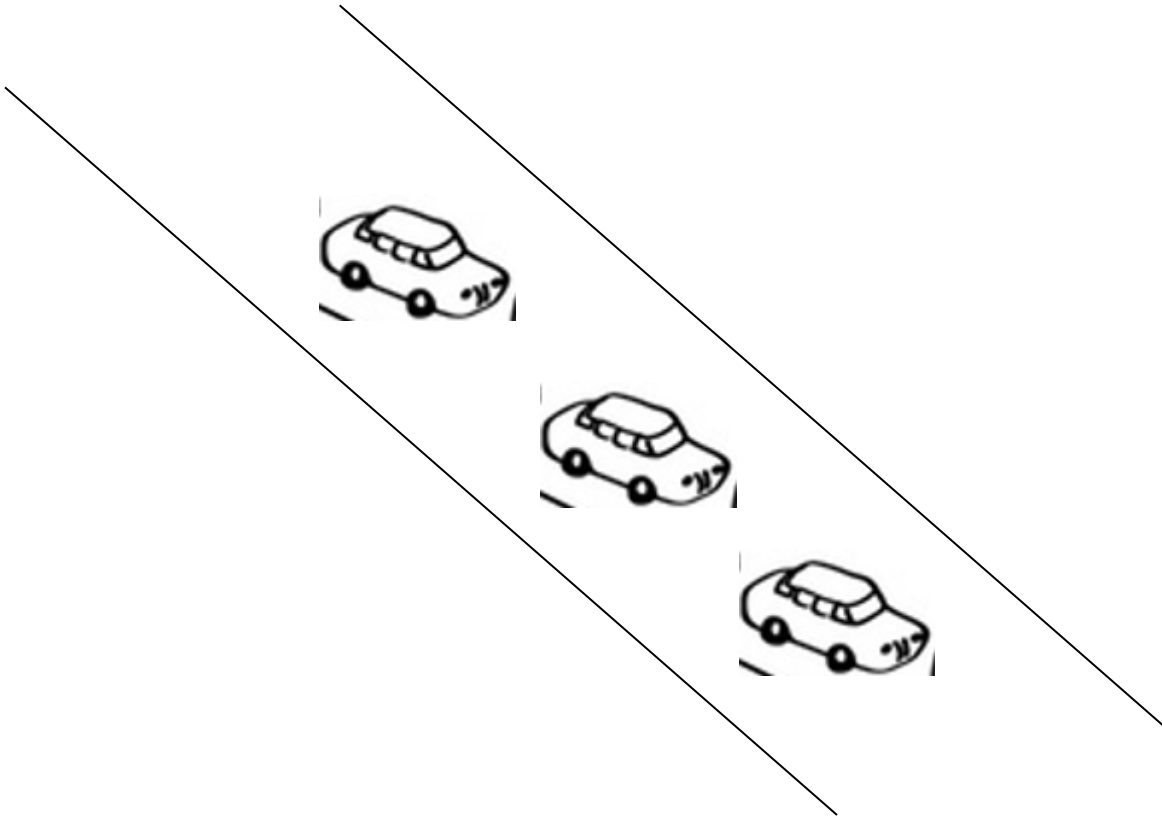
Cooperating Processes ?

Independent Process



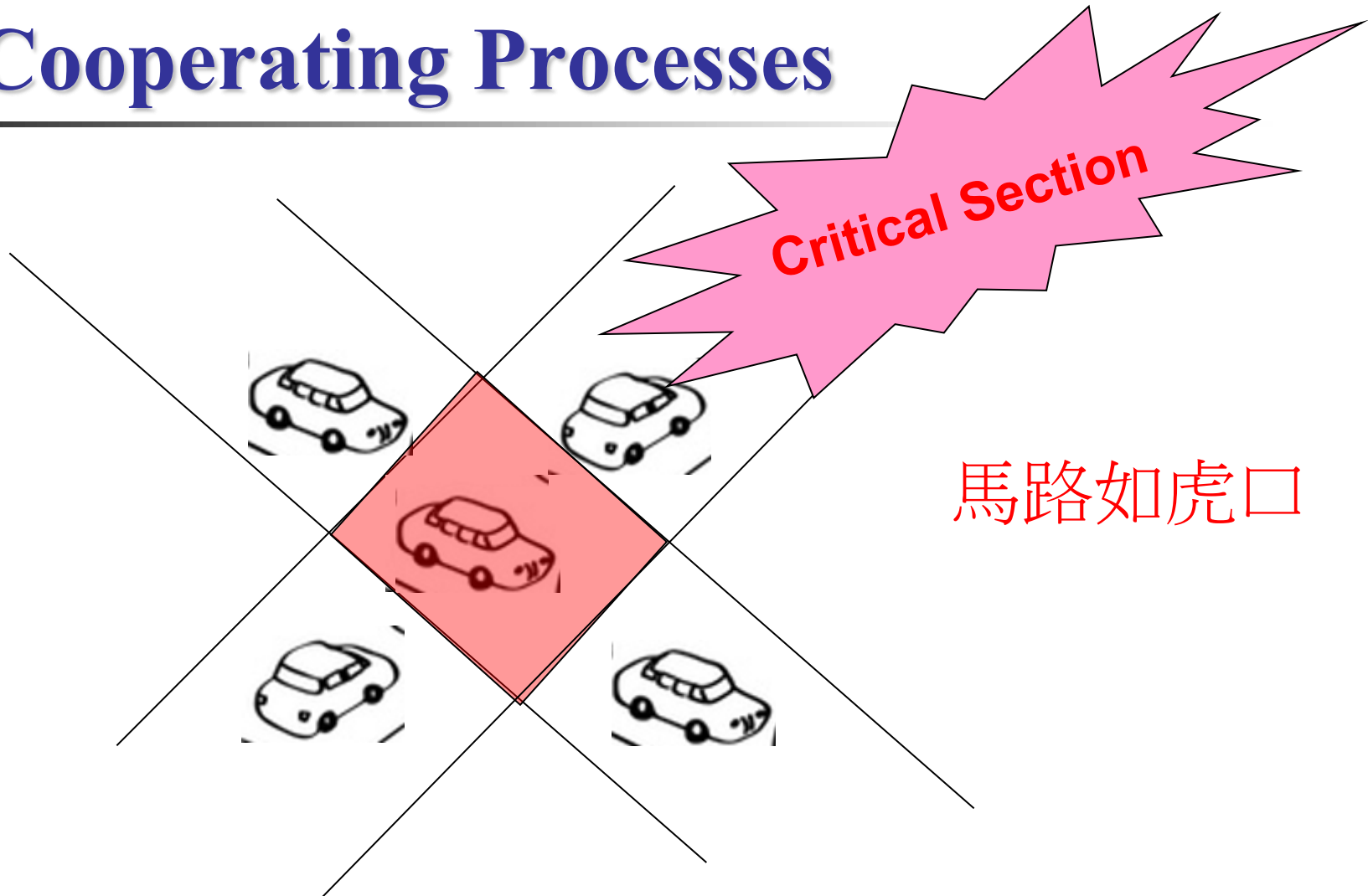


Independent Process



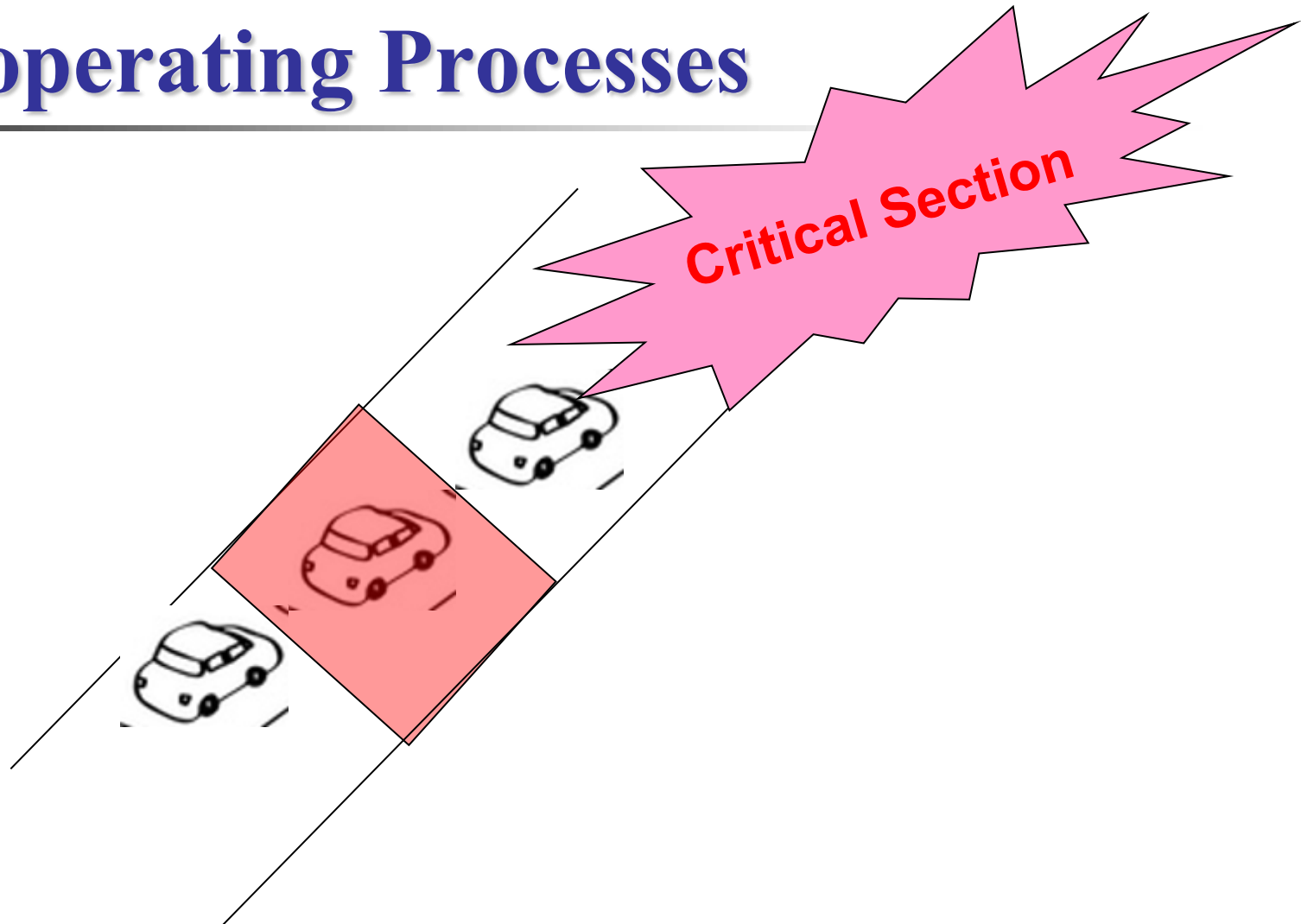
緊要的，關鍵性的，危急的

Cooperating Processes



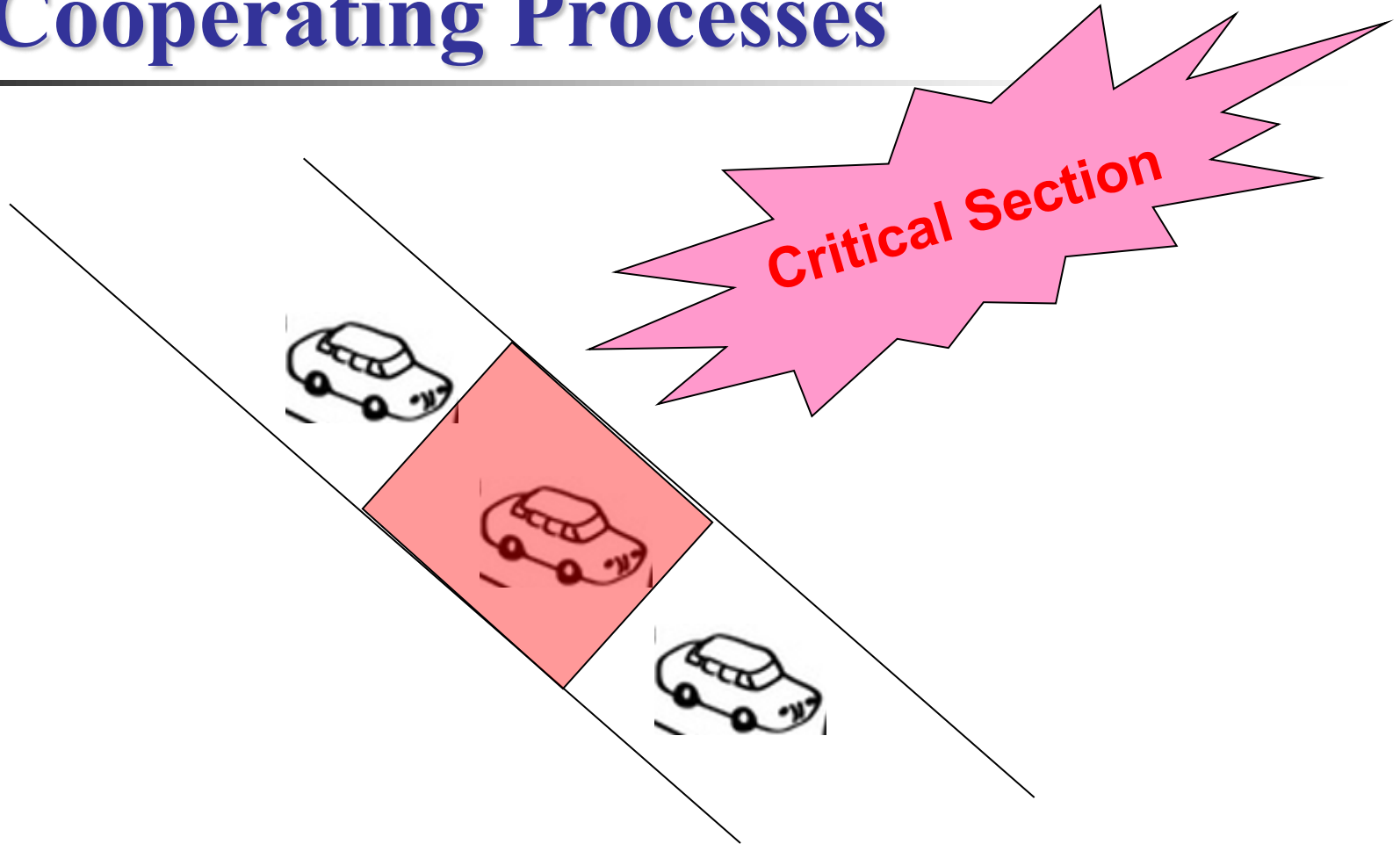
緊要的，關鍵性的，危急的

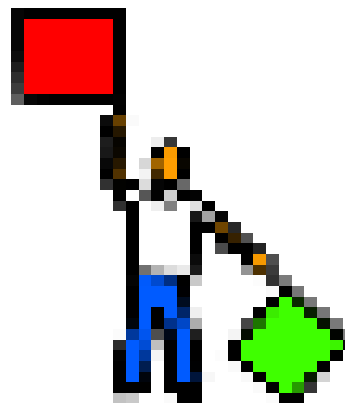
Cooperating Processes

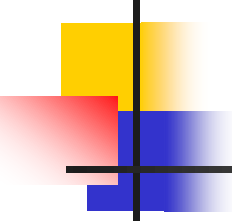


緊要的，關鍵性的，危急的

Cooperating Processes







Another Example

Bank account, **Saving** =100

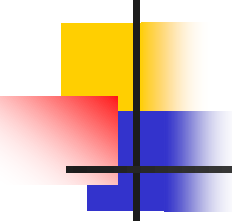
Deposit

Money=**Saving** (i.e., 100)

Money += 50

(now Money=150)

update Money to **Saving**



Another Example

Bank account, **Saving** =100

Withdraw

Money=**Saving** (i.e., 100)

Money -= 50

(now Money=50)

update Money to **Saving**

Another Example

Bank account, **Saving** =100

Deposit

Money=**Saving** (i.e., 100)
Money += 50
(now Money=150)

update Money to **Saving**

Withdraw

Money=**Saving** (i.e., 100)
Money -= 50
(now Money=50)

update Money to **Saving**

Concurrently access to shared data. Data is incorrect when operations are not atomic !!!

Another Example

Bank account, **Saving** = 150

Deposit

Money = **Saving** (i.e., 100)

Money += 50
(now Money = 150)

update Money to **Saving**

Withdraw

Money = **Saving** (i.e., 100)

Money -= 50
(now Money = 50)

update Money to **Saving**

Concurrently access to shared data. Data is incorrect when operations are not atomic !!!

Another Example

Bank account, **Saving** = 50

Deposit

Money=**Saving** (i.e., 100)

Money += 50
(now Money=150)

update Money to **Saving**

Withdraw

Money=**Saving** (i.e., 100)

Money -= 50
(now Money=50)

update Money to **Saving**

Concurrently access to shared data. Data is incorrect when operations are not atomic !!!

Another Example

Bank account, **Saving** = 50

Bank account, **Saving** = 150

Bank account, **Saving** = 100

Deposit

Money = **Saving** (i.e., 100)

Money += 50
(now Money = 150)

update Money to **Saving**

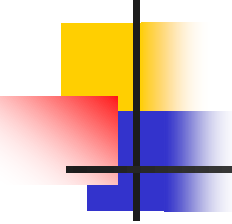
Withdraw

Money = **Saving** (i.e., 100)

Money -= 50
(now Money = 50)

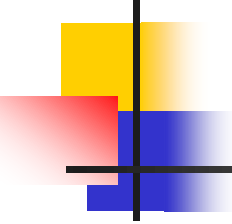
update Money to **Saving**

Concurrently access to shared data. Data is incorrect when operations are not atomic !!!



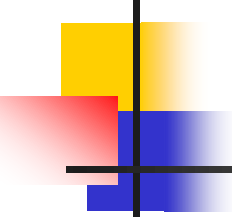
Chapter Objectives

- Describe the **critical-section problem** and illustrate a **race condition**
- Illustrate hardware solutions to the critical-section problem using **memory barriers**, **compare-and-swap operations**, and **atomic variables**
- Demonstrate how **mutex locks**, **semaphores**, **monitors**, and **condition variables** can be used to solve the critical section problem
- Evaluate tools that solve the critical-section problem in low-, moderate-, and high-contention scenarios



Chapter 6: Synchronization

- 6.1 Background 
- 6.2 The Critical-Section Problem 
- 6.3 Peterson's Solution 
- 6.4 Hardware Support for Synchronization 
- 6.5 Mutex Locks 
- 6.6 Semaphores 
- 6.7 Monitors 
- 6.8 Liveness 
- 6.9 Evaluation
- 6.10 Summary



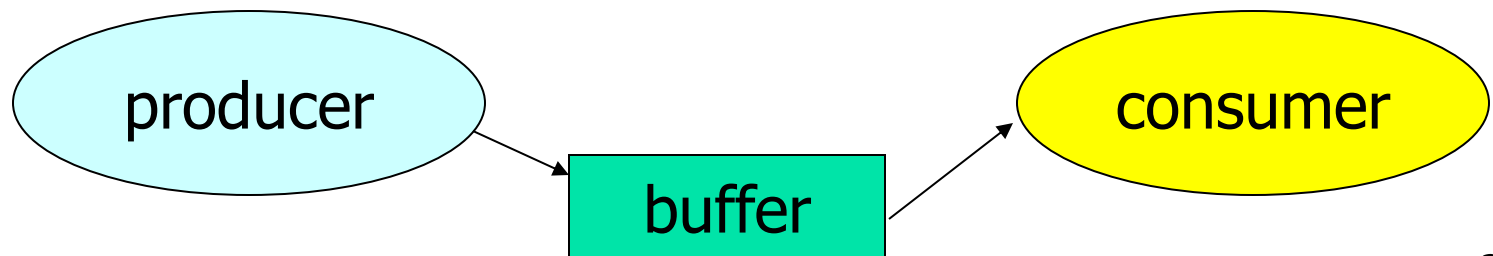
6.1 Background

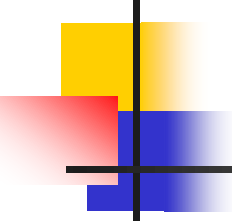
- A **cooperating process** can affect or be affected by other processes executing in the system
- The way that **cooperating processes** share data
 - directly share a logical address space (i.e., both code and data) (through the use of threads)
 - share data only through shared memory or message passing
- **Concurrent access** to shared data may result in **data inconsistency**
 - Need mechanisms to ensure the orderly execution of cooperating processes that share a logical address space
- Example
 - solution to the **producer-consumer problem**

Review:

Cooperating Processes

- Concurrent execution of cooperating processes requires mechanisms that allow processes to communicate with each other (Chapter 3) and to synchronize their actions (Chapter 6)
- Example: A **producer** process and a **consumer** process



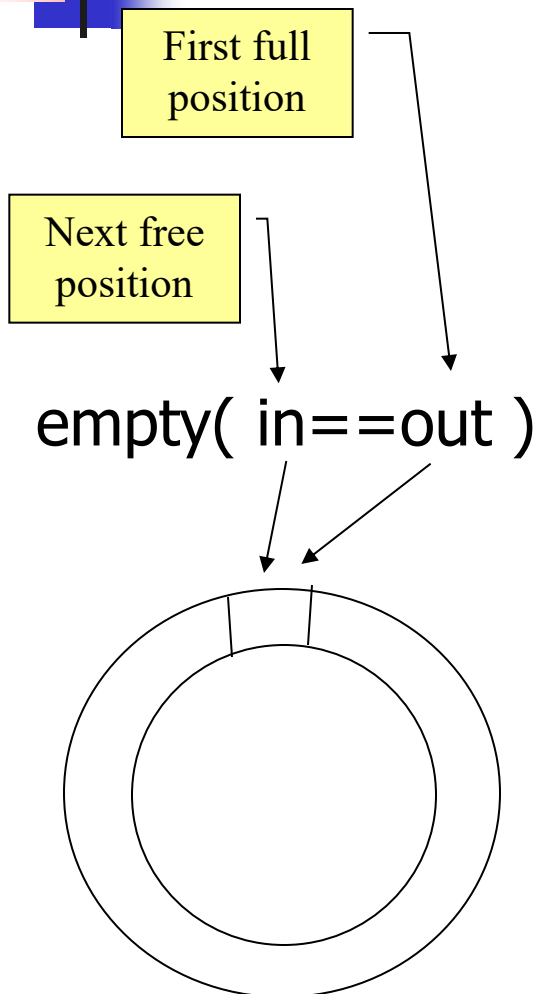


Producer-Consumer Problem

範例

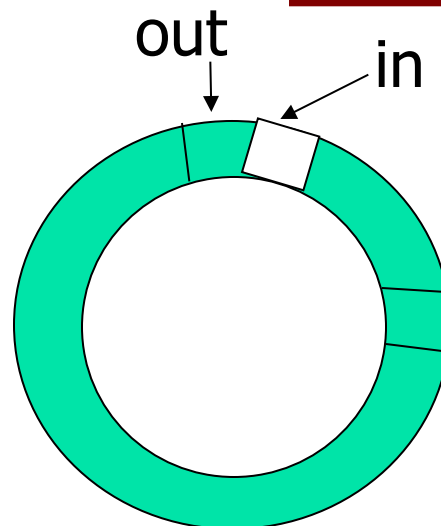
- Paradigm for cooperating processes, a **producer** process produces information that is consumed by a **consumer** process.
- The buffer may be either provided by OS through **the use of an IPC** facility, or by explicitly coded by application programmer with **the use of shared memory**.

Bounded-Buffer Shared-Memory Solution



■ Shared data

```
#define BUFFER_SIZE 10
typedef struct {
    ...
} item;
item buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
```



$((in+1) \% BUFFER_SIZE) == out$

A solution to the producer-consumer problem in Chapter 3

(Bounded-Buffer Shared-Memory Solution)

producer process

```
item nextProduced;  
  
while (true) {  
    while (((in + 1) % BUFFER_SIZE) == out)  
        ; /* do nothing */  
    buffer[in] = nextProduced;  
    in = (in + 1) % BUFFER_SIZE;  
}
```

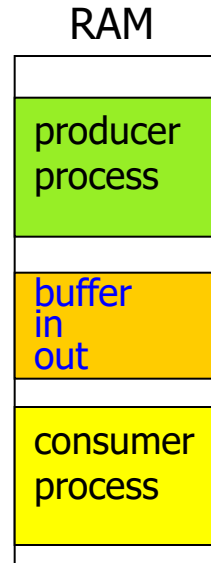
Buffer
is full

consumer process

```
item nextConsumed;  
  
while (true) {  
    while (in == out)  
        ; /* do nothing */  
    nextConsumed = buffer[out];  
    out = (out + 1) %  
        BUFFER_SIZE;  
}
```

Buffer is
empty

Bounded-Buffer Shared-Memory Solution (Cont.)



■ Shared data

```
#define BUFFER_SIZE 10
typedef struct {
    ...
} item;
item buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
```

■ Producer process

```
item nextProduced;

while (true) {
    /* produce an item in nextProduced */
    while (((in + 1) % BUFFER_SIZE) == out) // full
        ; /* do nothing */
    buffer[in] = nextProduced;
    in = (in + 1) % BUFFER_SIZE;
}
```

■ Consumer process

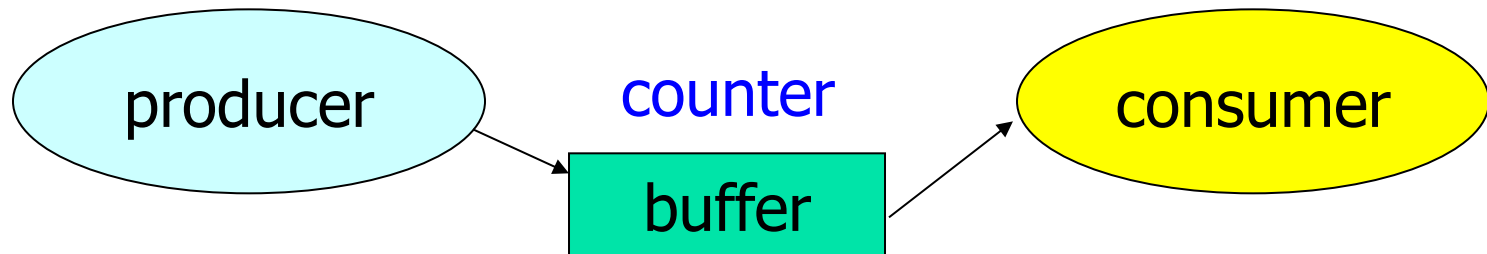
```
item nextConsumed;

while (true) {
    while (in == out) // empty
        ; /* do nothing */
    nextConsumed = buffer[out];
    out = (out + 1) % BUFFER_SIZE;
    /* consume the item in nextConsumed */
}
```

Bounded-Buffer

Shared-Memory Solution (Cont.)

- The solution in Chapter 3 allows at most ***BUFFERSIZE-1*** items in the buffer at the same time.
- To have ***BUFFERSIZE*** items, we can use a "***counter***".
 - Initialize to 0, and increased every time when a new item is added into the buffer, and decreased when removed



Another solution to the producer-consumer problem

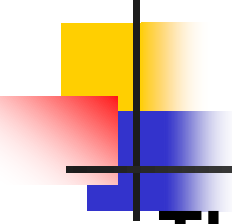
(Using a shared **counter**)

producer process

```
item nextProduced;  
  
while (true) {  
    /* produce an item in nextProduced*/  
    while (counter == BUFFER_SIZE)  
        ; /* do nothing */  
    buffer[in] = nextProduced;  
    in = (in + 1) % BUFFER_SIZE;  
    counter++;  
}
```

consumer process

```
item nextConsumed;  
  
while (true) {  
    while (counter == 0)  
        ; /* do nothing */  
    nextConsumed = buffer[out];  
    out = (out + 1) % BUFFER_SIZE;  
    counter--;  
    /* produce an item in nextProduced*/  
}
```

Incorrect State

- Though both the producer and consumer routines are correct separately, **they may not function correctly when executed concurrently.**
- The statements may be implemented in machine language as:

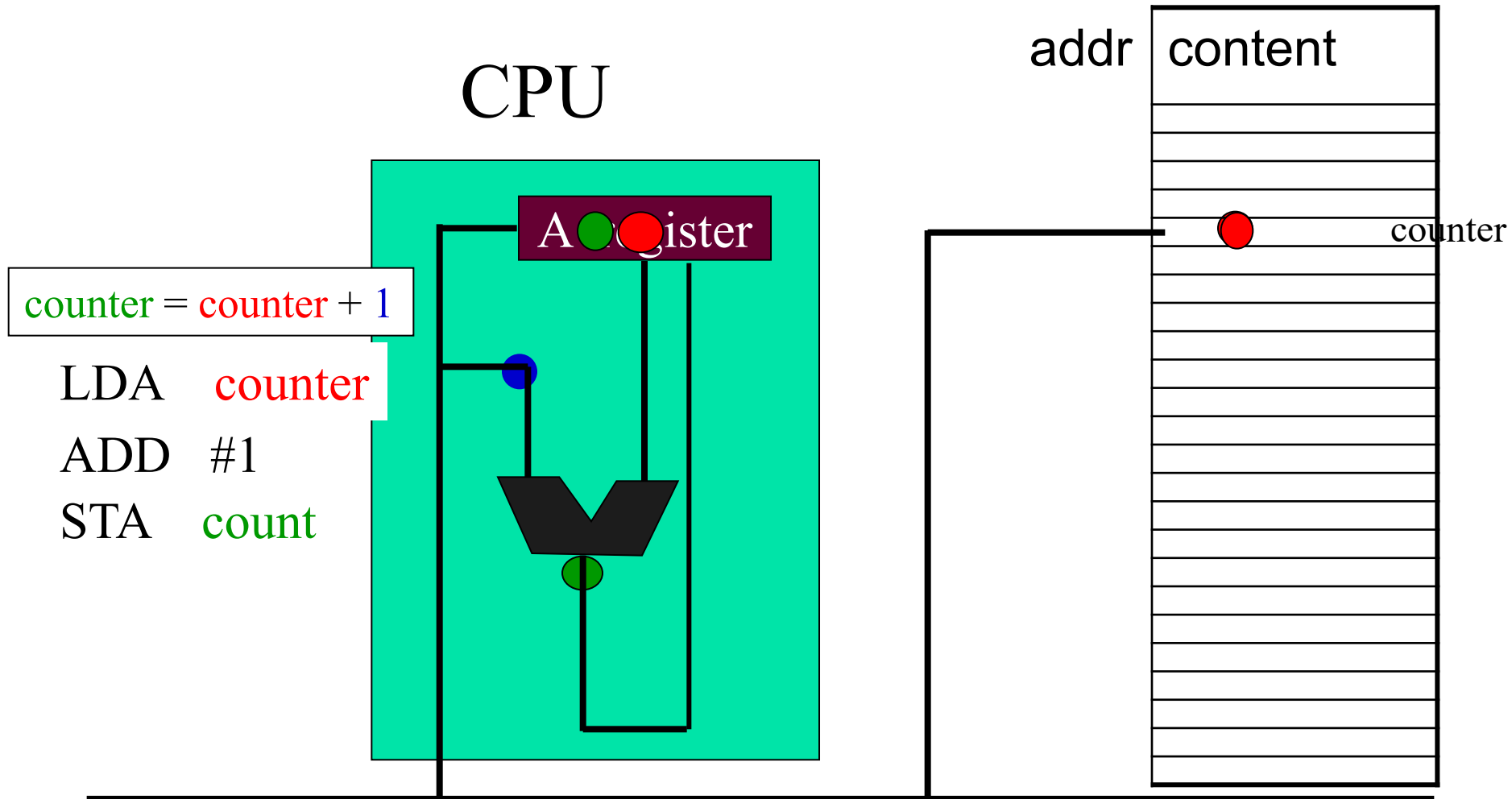
counter ++

```
register1 = counter  
register1 = register1 + 1  
counter = register1
```

counter --

```
register2 = counter  
register2 = register2 - 1  
counter = register2
```

Arithmetic : ADD 、 SUB 、 MUL 、 DIV



counter ++

```
register1 = counter  
register1 = register1 + 1  
counter = register1
```

counter --

```
register2 = counter  
register2 = register2 - 1  
counter = register2
```

■ What happens if counter = 5 ?

T0:	register ₁ = counter	{register ₁ = 5}	
T1:	register ₁ = register ₁ + 1	{register ₁ = 6}	
T2:		{register ₂ = 5}	register ₂ = counter
T3:		{register ₂ = 4}	register ₂ = register ₂ - 1
T4:	counter = register ₁	{counter = 6}	
T5:		{counter = 4}	counter = register ₂

Correct answer should be
counter = 5.

Incorrect state !!!

counter ++

register₁ = counter
register₁ = register₁ + 1
counter = register₁

counter --

register₂ = counter
register₂ = register₂ - 1
counter = register₂

■ What happens if counter = 5 ?

T0: *register₁ = counter* {*register₁ = 5*}

T1: *register₁ = register₁ + 1* {*register₁ = 6*}

T2: {*register₂ = 5*}

T3: {*register₂ = 4*}

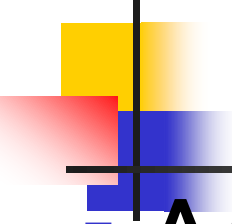
T4: {*counter = 4*}

T5: *counter = register₁* {*counter = 6*}

register₂ = counter
register₂ = register₂ - 1
counter = register₂

Correct answer should be
counter = 5.

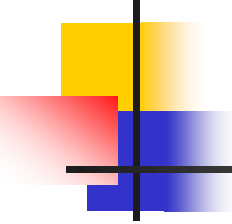
Incorrect state !!!



Race Condition

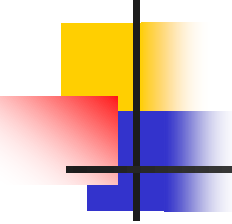
- A situation like this, where several processes **access** and **manipulate** the same data **concurrently**, and the outcome of the execution depends on the particular **order** in which the **access** takes place, is called a **race condition**
- To guard against the race condition
 - Ensure only one process at a time can be manipulating shared data (i.e., the variable counter).
 - process **synchronization** and **coordination** among cooperating processes



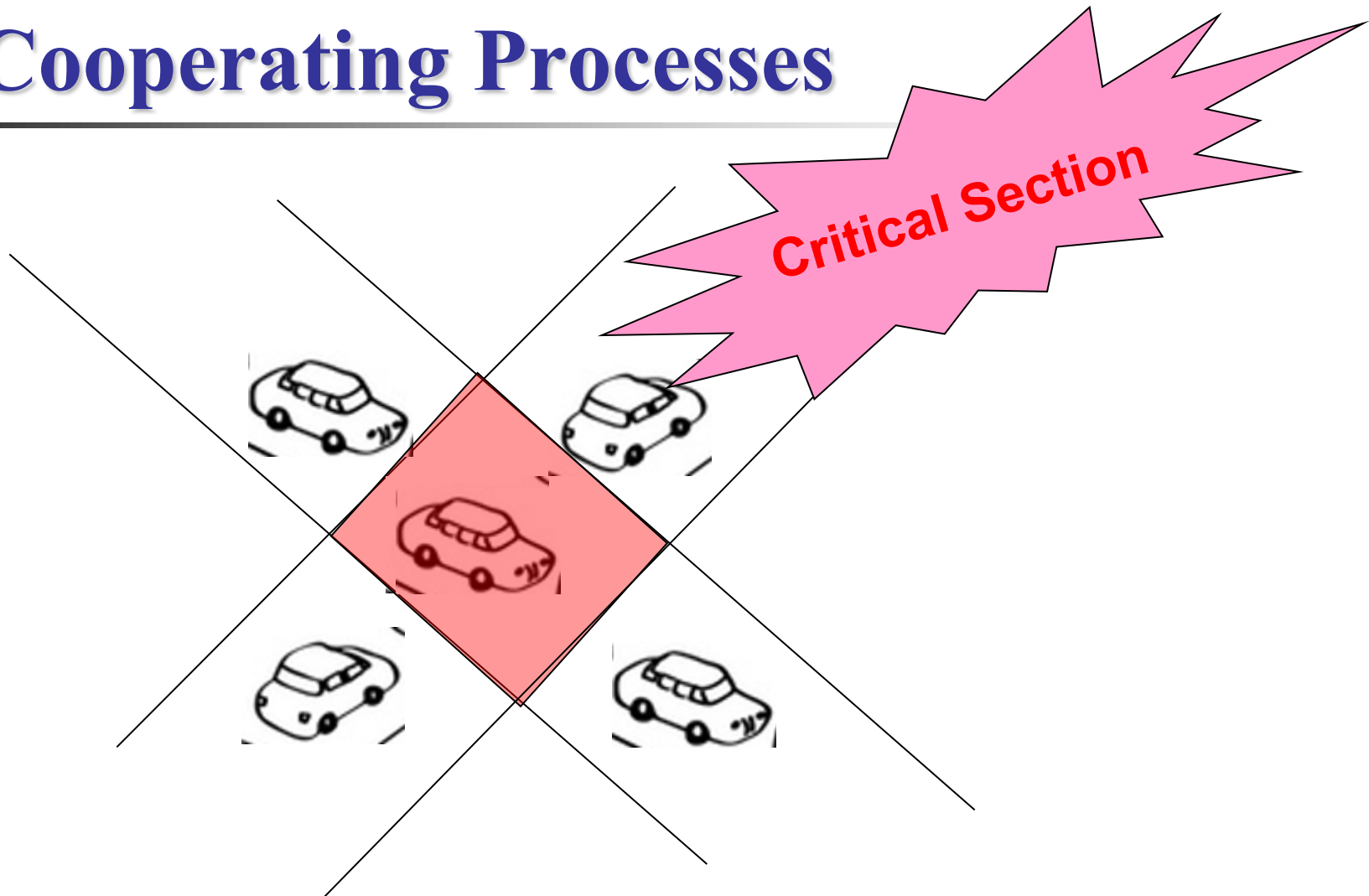


6.2 The Critical-Section Problem

- Consider system of n processes $\{p_0, p_1, \dots, p_{n-1}\}$
- Each process has a segment of code, called a critical section, in which the process may be accessing -- and updating -- data that is shared with at least one other process.
- Problem – ensure that when a process is executing in its critical section, no other process is allowed to execute in its critical section.
 - No two processes are executing in their critical sections at the same time.

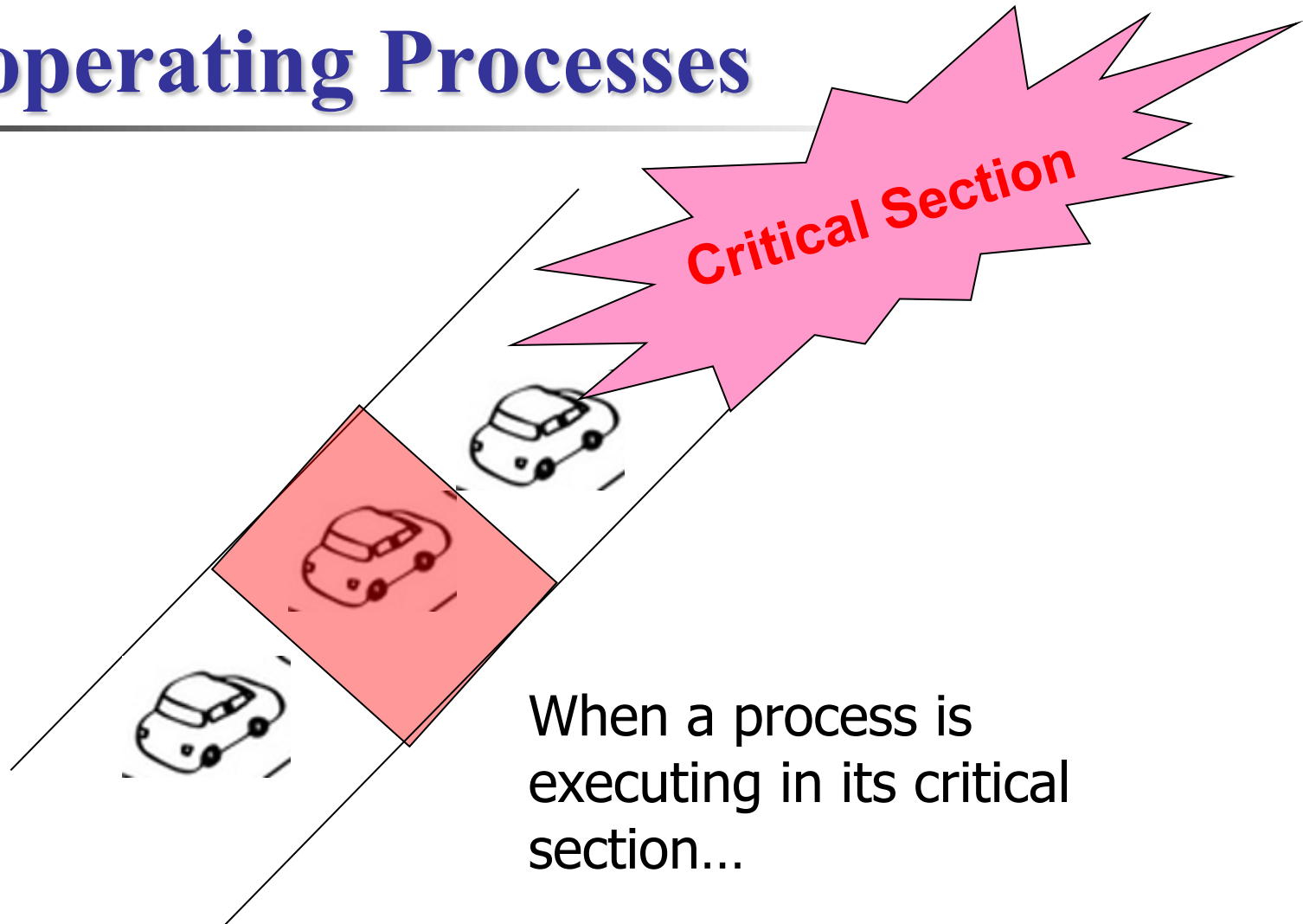


Cooperating Processes



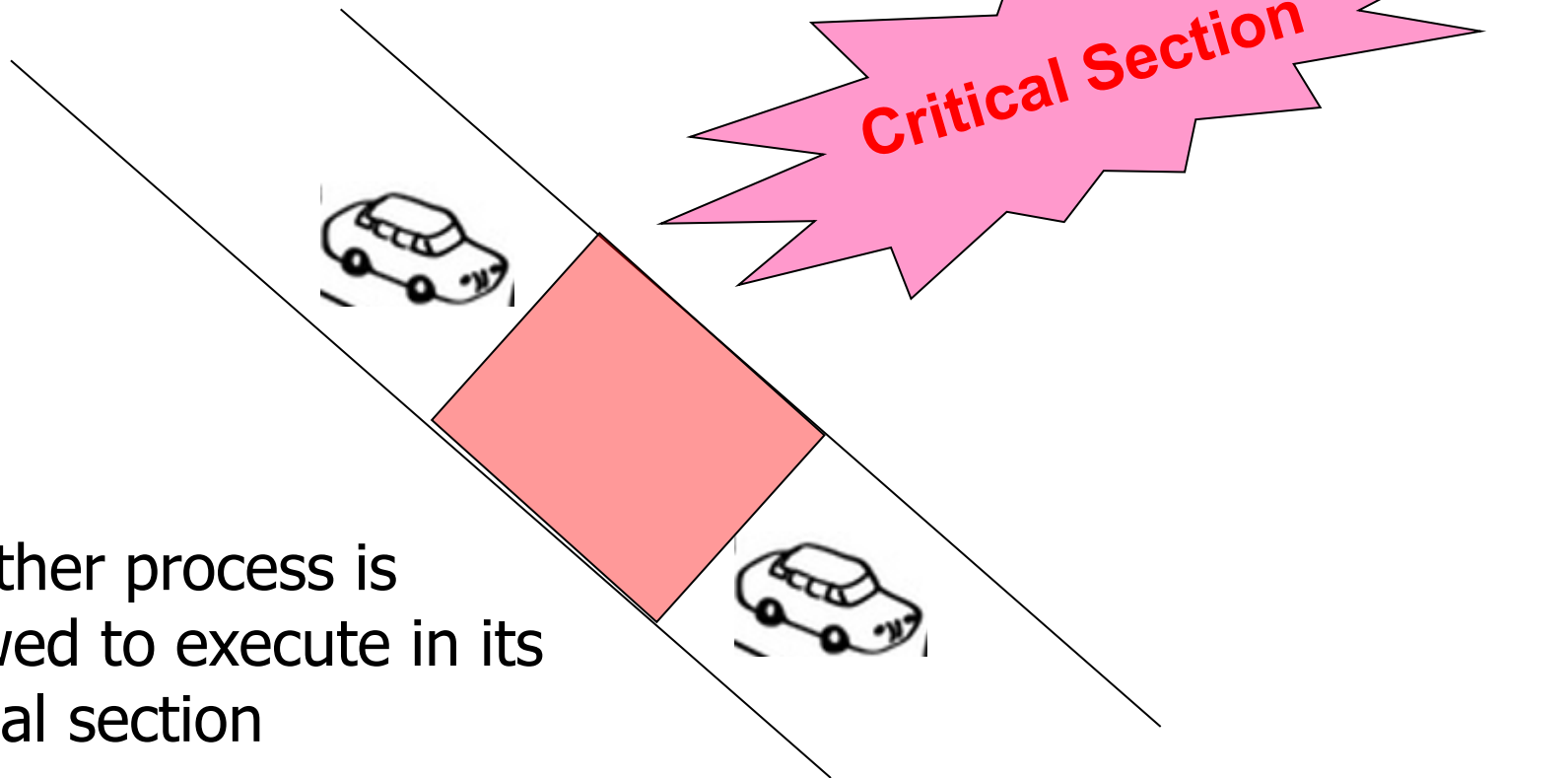
緊要的，關鍵性的，危急的

Cooperating Processes



緊要的，關鍵性的，危急的

Cooperating Processes

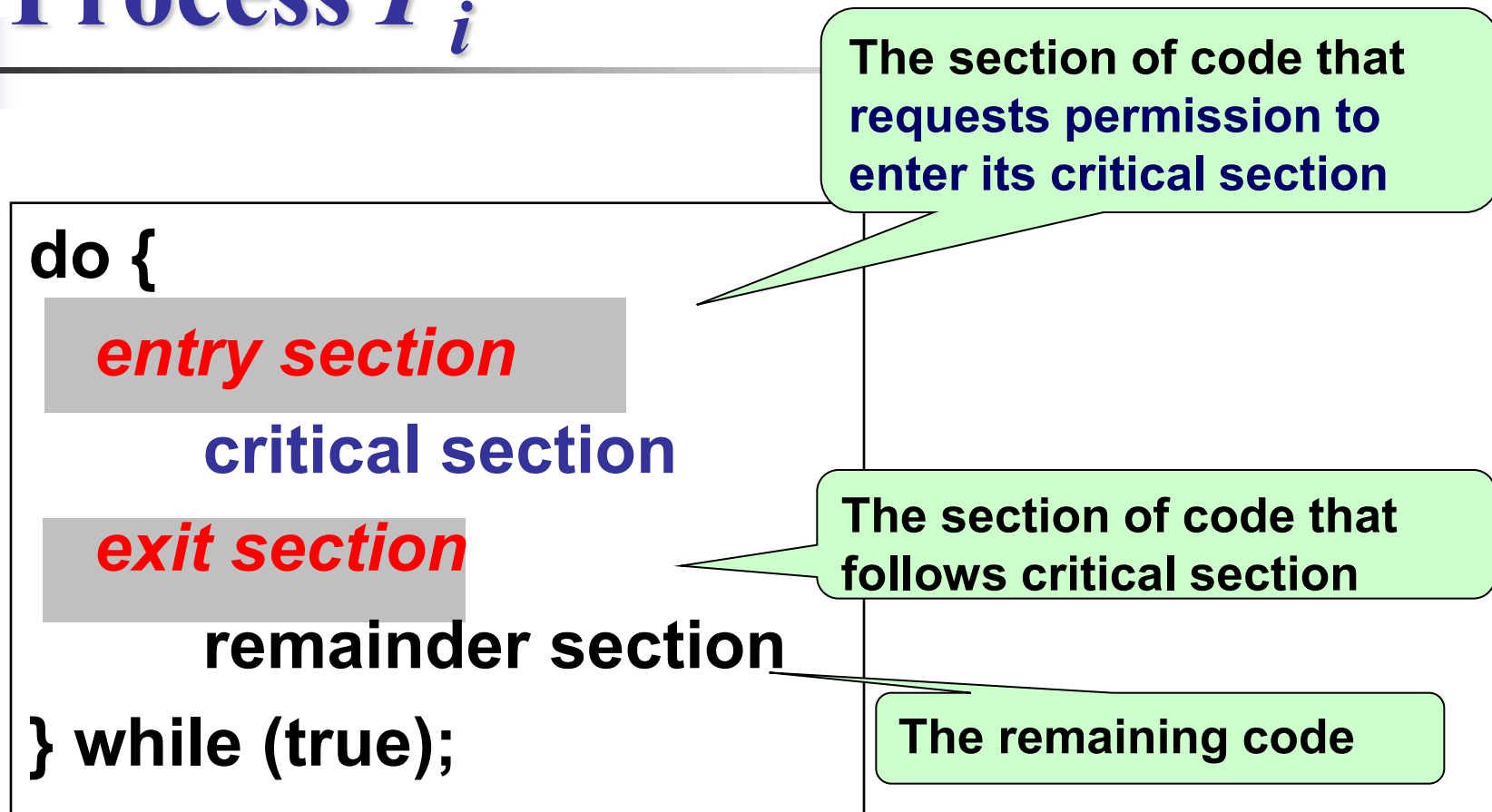




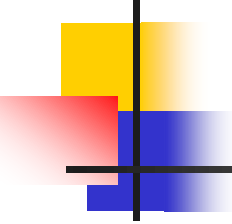
Critical-Section Problem (Cont.)

- The critical-section problem is to design a **protocol** that the processes can use to synchronize their activity so as to cooperatively share data.
- **Only one process** at a time can be executing its critical section, that is, the *execution* of critical sections is ***mutually exclusive***.

General Structure of a Typical Process P_i



Process P_i



Solution Requirements

- **Mutual Exclusion**

- If P_i is executing in its CS, no other P s can be executing in their CSs.

- **Progress** 進行,進展

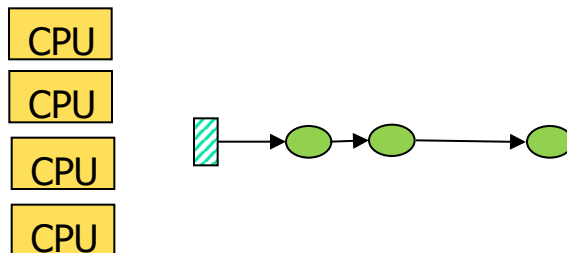
- If no P is executing in its CS and some P s wish to enter their CSs, eventually one P will be selected and get in (not prevented by someone who *doesn't* want in)

- **Bounded Waiting**

- Exist a bound, or limit, on the number of times to grant a P 's request to enter its CS.
- No assumption concerning **relative speed** of the n processes

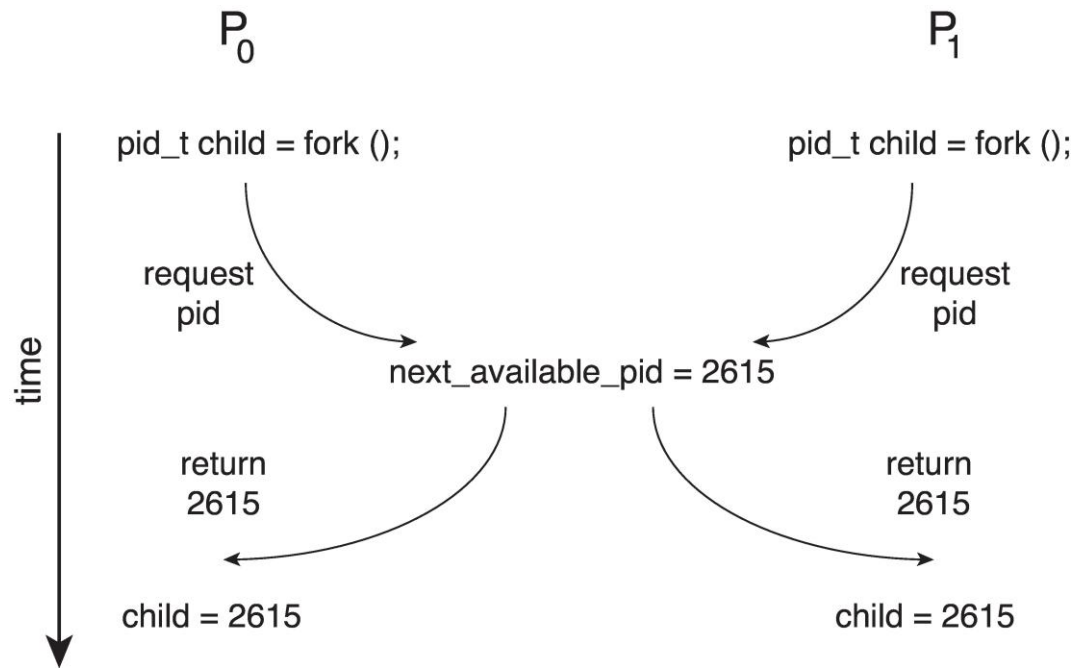
Critical-Section Problem (Cont.)

- **At a given point in time, many kernel-mode processes may be active in OS.**
 - **Kernel code is subject to several possible race conditions**
 - For accessing open file list, data structures for maintaining memory allocation, process lists, interrupt handling.
 - **It is up to kernel developers to ensure that the OS is free from such race conditions.**



Race Condition Example

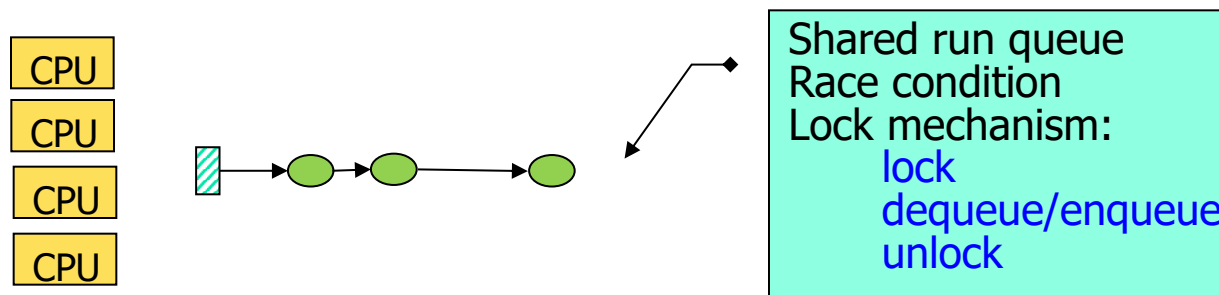
- Processes P_0 and P_1 are creating child processes using the `fork()` system call
- Race condition on kernel variable `next_available_pid` which represents the next available process identifier (pid)
- Unless there is mutual exclusion, the same pid could be assigned to two different processes!

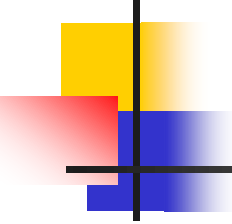


Two general approaches used to handle critical sections in OS

■ (1) preemptive kernels

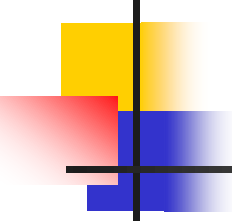
- Allows a process to be preempted while it is running in kernel mode.
- Must be carefully designed to ensure that shared kernel data are free from race conditions.





Two general approaches used to handle critical sections in OS (Cont.)

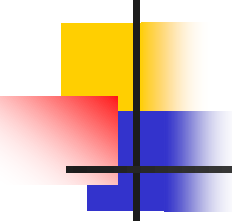
- **(2) nonpreemptive kernels**
 - Does not allow a process running in kernel mode to be preempted
 - A kernel-mode process will run until it exits kernel mode, blocks, or voluntarily yields control of the CPU
 - Essentially free from race conditions on kernel data structures, as only one process is active in the kernel at a time.



Why would anyone favor a preemptive kernel over a nonpreemptive one?

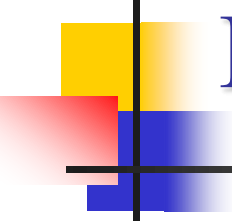
- **A preemptive kernel may be more responsive**
 - Since less risk that a kernel-mode process will run for an arbitrarily long period before relinquishing the processor to waiting processes.
- **A preemptive kernel is more suitable for real-time programming**
 - As it will allow a real-time process to preempt a process currently running in the kernel.





6.3 Peterson's Solution

- **Peterson's solution: a classic software-based solution to the critical-section problem**
 - Restricted to two processes that ^{使交替, 使輪流} alternate execution between their critical sections and remainder sections.



Peterson's Solution (Cont.)

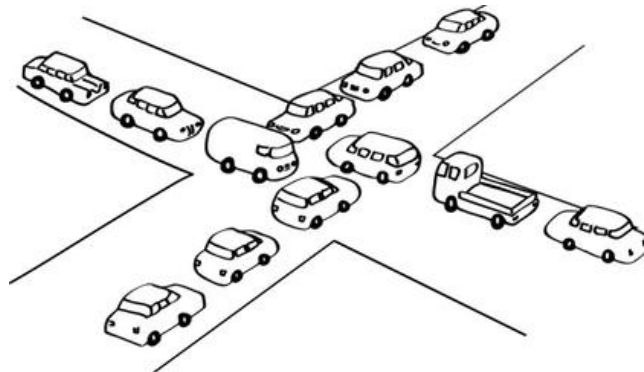
- Only 2 processes: P_0 and P_1
- General structure of process P_i (other process P_j)

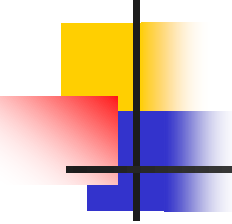
```
do {  
    entry section  
    critical section  
    exit section  
    remainder section  
} while (1) ;
```

- Processes may share some common variables to synchronize their actions.

Peterson's Solution (Cont.)

- Require the two processes to share two variables to synchronize their actions :
 - `int turn;` // indicates whose turn to enter critical section
 - `boolean flag[2];`
// indicate if a process is *ready* to enter its critical section.
// `flag[i] = true` implies that process P_i is ready!





Peterson's Solution (Cont.)

- Shared data:

boolean flag[2]

int turn;

- $\text{flag}[0] = \text{flag}[1] = \text{false};$
 $\text{turn} = 0 \text{ or } 1, \text{ initially}$

```
while (TRUE) {  
    flag[i] = TRUE;  
    turn = j;  
    while (flag[j] && turn == j)  
        ;  
    // critical section  
    flag[i] = FALSE;  
    // remainder section  
}
```

Process P_i

All three requirements
are met !!!

Peterson's Solution (Cont.)

- Shared data:
boolean flag[2]
int turn;

```
while (TRUE) {  
    flag[0] = TRUE;  
    turn = 1;  
    while (flag[1] && turn == 1)  
        ;
```

// critical section

```
flag[0] = FALSE;
```

// remainder section

```
}
```

Process P_0

```
while (TRUE) {  
    flag[1] = TRUE;  
    turn = 0;  
    while (flag[0]) && (turn == 0)  
        ;
```

// critical section

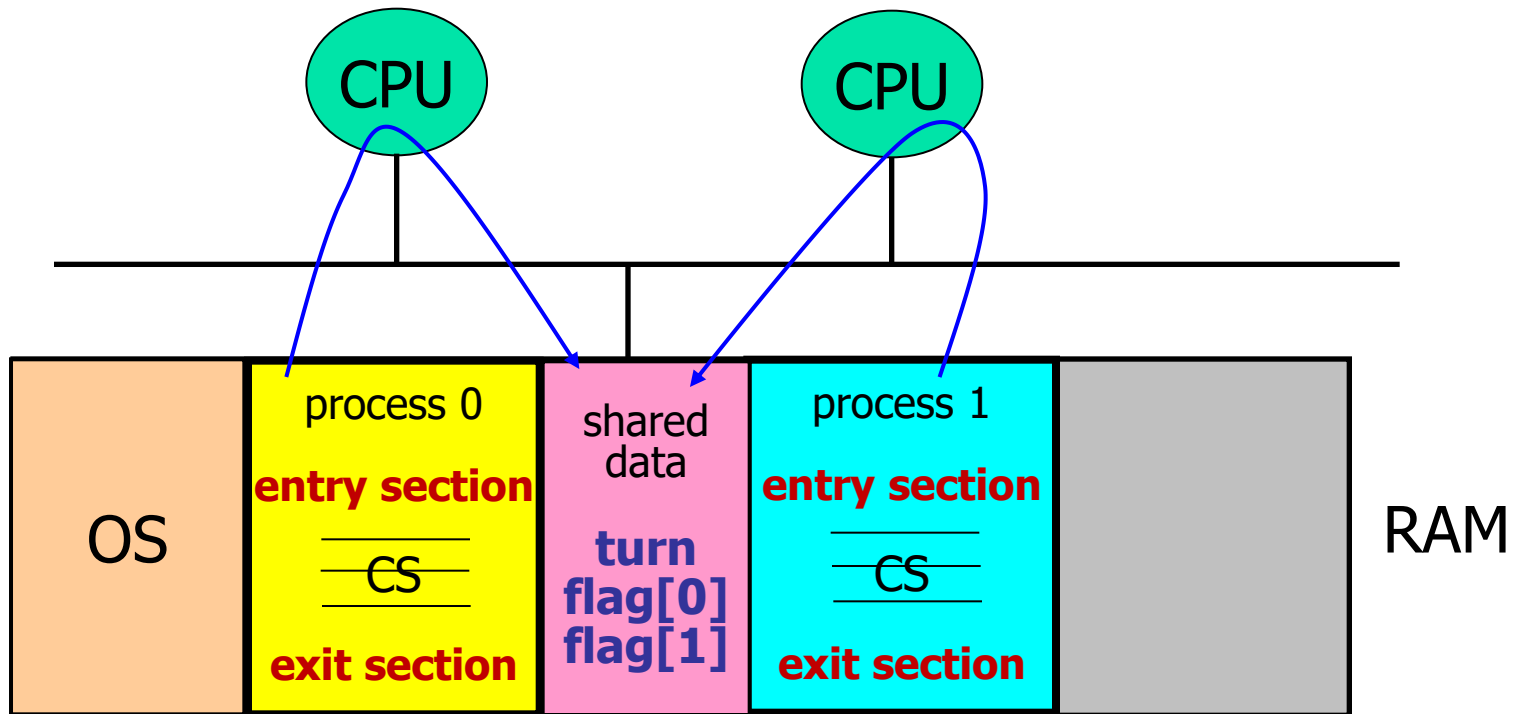
```
flag[1] = FALSE;
```

// remainder section

```
}
```

Process P_1

Multiprocessor (Peterson's solution for 2 processes)



Peterson's Solution (Cont.)

■ Proof mutual exclusion:

- P_i in CS only if $flag[j] == FALSE$ or $turn == i$
- If P_0 (or P_1) has already in CS, P_1 (or P_0) wants to enter CS, (though $flag[0] = flag[1] = TRUE$, but $turn = 0$ (or $turn = 1$))
 - P_1 (or P_0) must wait
- if both P_0 and P_1 want to enter CS ($flag[0] = flag[1] = TRUE$)
 - $turn = 0 \Rightarrow P_1$ looping or $turn = 1 \Rightarrow P_0$ looping

$flag[0]$	$flag[1]$	$turn$	P_0	P_1
1	1	0	in CS	in looping
1	1	1	in looping	in CS

Peterson's Solution (Cont.)

■ Proof progress:

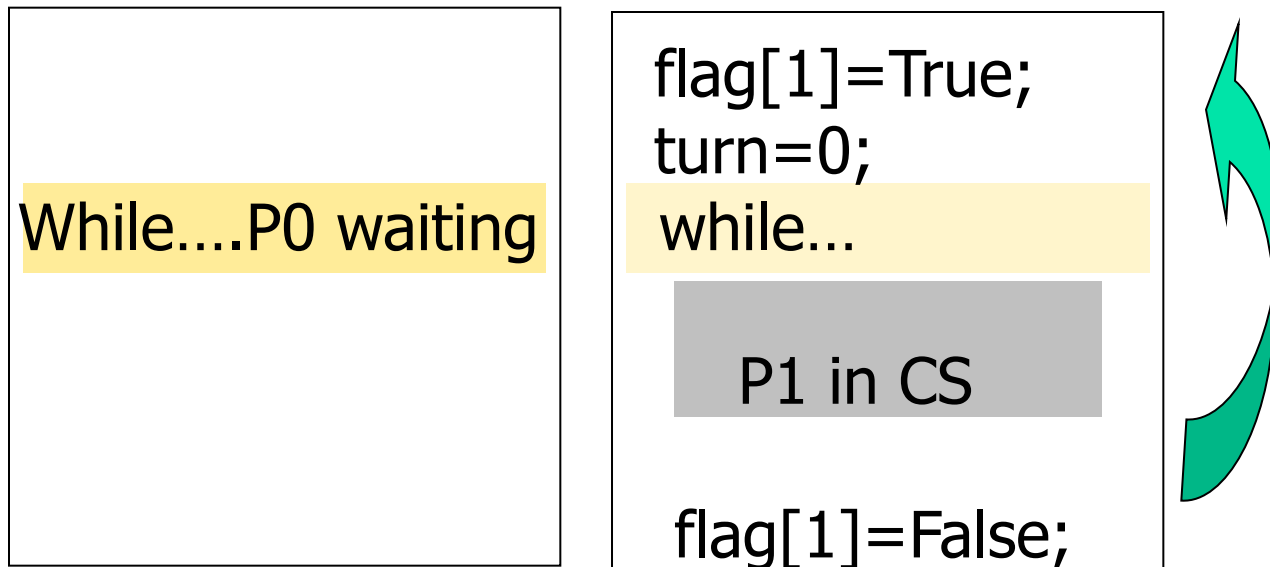
- P_i stuck in while loop only if **$flag[j]=true$** and **$turn=j$** ,
- if P_j is not ready to enter CS, $flag[j]=false$, then P_i can enter its CS
- if P_i and P_j all want to enter CS, then who can enter CS is determined by turn.
 - e.g., if $turn=i$, P_i in CS and P_j in while loop

Once P_i exits CS, then setting $flag[i]=false$ to allow P_j to enter CS

$flag[0]$	$flag[1]$	$turn$	P_0	P_1
0	1		out CS	in CS
1	0		in CS	out CS
1	1	0	in CS	in looping
1	1	1	in looping	in CS

Peterson's Solution (Cont.)

- **Proof bounded-waiting:**
 - if P_i in while loop and if P_j resets $\text{flag}[j]=\text{true}$ and $\text{turn}=i$, then P_i will enter CS (progress) after at most one entry by P_j (bounded waiting).



About Peterson's Solution (Cont.)

In each process_{*i*}, setting
turn=**j** before entering CS;

do {

flag[**i**] = true;

turn = **j**;

while (*flag*[**j**]) && (turn
== **j**) ;

critical section

flag[**i**] = false;

remainder section

} while (1) ;

Process *P_i*

What if
in each process_{*i*}, setting
turn=**i** before entering CS

do {

flag[**i**] = true;

turn = **i**;

while (*flag*[**j**]) && (turn
== **j**) ;

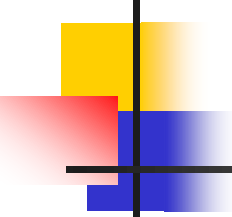
critical section

flag[**i**] = false;

remainder section

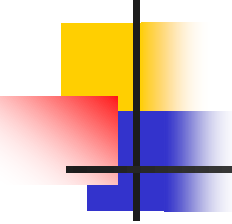
} while (1) ;

Process *P_i*



Peterson's Solution (Cont.)

- Although useful for demonstrating an algorithm, Peterson's Solution is not guaranteed to work on modern architectures.
- Understanding why it will not work is also useful for better understanding race conditions.
- To improve performance, processors and/or compilers may reorder operations that have no dependencies.
- For single-threaded application, this is ok as the result will always be the same.
- For a multithreaded application, the reordering may produce inconsistent or unexpected results!



Peterson's Solution (Cont.)

- Two threads share the data:

```
boolean flag = false;  
int x = 0;
```

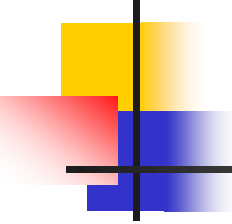
- Thread 1 performs

```
while (!flag)  
    ;  
print x
```

- Thread 2 performs

```
x = 100;  
flag = true
```

- What is the expected output?



Peterson's Solution (Cont.)

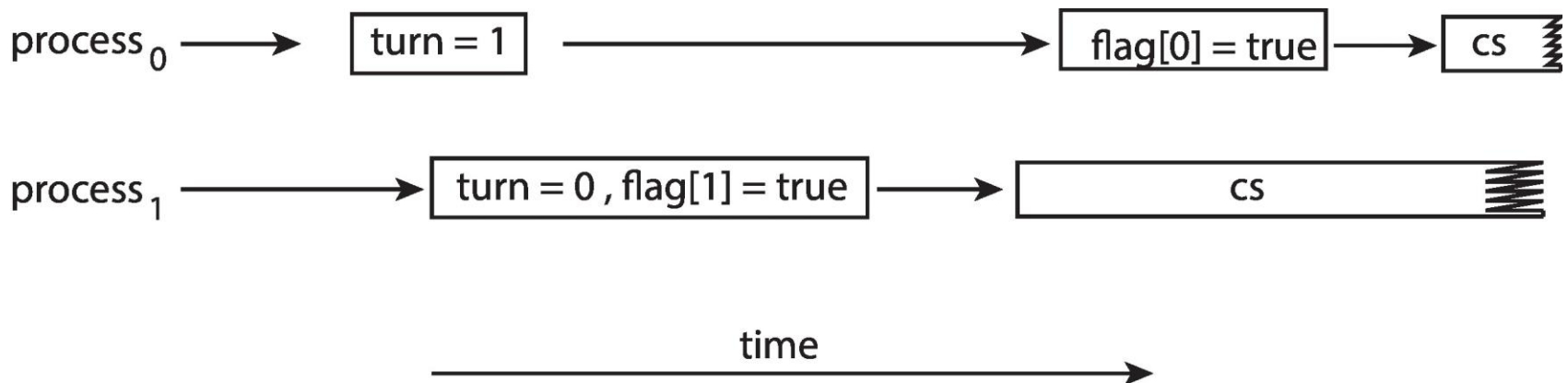
- 100 is the expected output.
- However, the operations for Thread 2 may be reordered:

```
flag = true;  
x = 100;
```

- If this occurs, the output may be 0!

Peterson's Solution (Cont.)

- The effects of instruction reordering in Peterson's Solution



- This allows both processes to be in their critical section at the same time!

Peterson's Solution (Cont.)

■ Shared data:
boolean flag[2]
int turn;

```
while (TRUE) {
```

```
    turn = 1; ①
```

```
    flag[0] = TRUE; ⑥
```

```
    while (flag[1] && turn == 1)
```

```
        ;
```

```
        // critical section ⑧
```

```
    flag[0] = FALSE;
```

```
    // remainder section
```

```
}
```

Process P_0

```
while (TRUE) {
```

```
    turn = 0; ②
```

```
    flag[1] = TRUE; ③
```

```
    while (flag[0]) && (turn == 0)
```

```
        ;
```

```
        // critical section ⑤
```

```
    flag[1] = FALSE;
```

```
    // remainder section
```

```
}
```

Process P_1

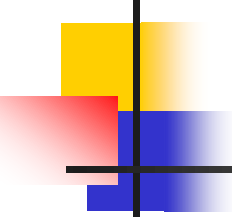




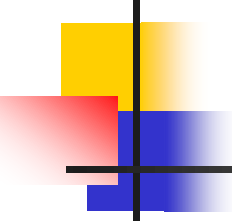
Peterson's solution 補充說明

- Peterson's solution is based on the software to solve the critical section problem. Because of the way modern computer architectures perform basic machine language instruction such as load and store, there are no guarantees that Peterson's solution will work on such architectures.
- 在 Multicore multiprocessors 下, CPU may reorder memory access, 要加一些 memory barrier (processor instruction, (aka. memory fences, mfences))才能確保 memory access 的順序不被 CPU reorder.
- <http://bartoszmilewski.com/2008/11/05/who-ordered-memory-fences-on-an-x86/>
- <http://bartoszmilewski.com/2008/08/04/multicores-and-publication-safety/>

6.4 Hardware Support for Synchronization



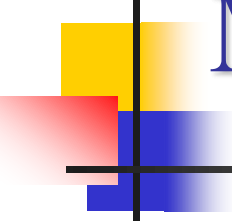
- Many systems provide hardware support for implementing the critical section code.
- Uniprocessors – could disable interrupts
 - Currently running code would execute without preemption
 - Generally too inefficient on multiprocessor systems
- Three forms of hardware support:
 - 6.4.1 Memory barriers
 - 6.4.2 Hardware instructions
 - 6.4.3 Atomic variables



6.4.1 Memory Barriers

For shared-memory multiprocessor

- **Memory models are the memory guarantees a computer architecture makes to application programs.**
- **Memory models may be either:**
 - **Strongly ordered** – where a memory modification of one processor is immediately visible to all other processors.
 - **Weakly ordered** – where a memory modification of one processor may not be immediately visible to all other processors.
- **A **memory barrier** is an instruction that forces any change in memory to be propagated (made visible) to all other processors.**



Memory Barrier

- We could add a **memory barrier** to the following instructions to ensure Thread 1 outputs 100:
- Thread 1 now performs

```
while (!flag)
    memory_barrier();
print x
```

- Thread 2 now performs

```
x = 100;
memory_barrier();
flag = true
```

Peterson's Solution (Cont.)

■ Shared data:

boolean flag[2]

int turn;

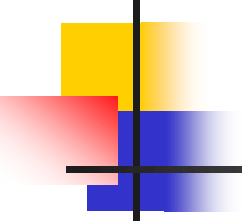
- $\text{flag}[0] = \text{flag}[1] = \text{false};$
 $\text{turn} = 0 \text{ or } 1, \text{ initially}$

Add memory barrier to
force memory
modifications are
visible to all processors.

```
while (TRUE) {  
    flag[i] = TRUE;  
    memory_barrier();  
    turn = j;  
    while (flag[j] && turn == j)  
        ;  
    // critical section  
    flag[i] = FALSE;  
    // remainder section  
}
```

Process P_i

Peterson's Solution (Cont.)



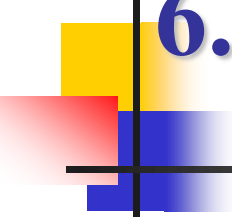
- Shared data:
boolean flag[2]
int turn;

```
while (TRUE) {  
    flag[0] = TRUE;  
    memory_barrier();  
    turn = 1;  
    while (flag[1] && turn == 1)  
        ;  
    // critical section  
    flag[0] = FALSE;  
    // remainder section  
}
```

Process P_0

```
while (TRUE) {  
    flag[1] = TRUE;  
    memory_barrier();  
    turn = 0;  
    while (flag[0]) && (turn == 0)  
        ;  
    // critical section  
    flag[1] = FALSE;  
    // remainder section  
}
```

Process P_1



6.4.2 Hardware Instructions

- Some simple **hardware instructions** available on many systems can be used effectively in solving critical section problems.
- Modern computer systems provide special **atomic hardware instructions: (one uninterruptible unit)**
 - **test_and_set()**
 - **compare_and_swap()**

test_and_set()

Atomic Instruction

```
boolean lock1=FALSE;  
boolean lock2=FALSE;  
lock2 =test_and_set(&lock1);  
printf("%d %d", lock1, lock2 );
```

T F

- Test and modify the content of a word atomically.

```
boolean test_and_set(boolean *target)  
{  
    boolean rv = *target;  
    *target = TRUE;  
    return rv;  
}
```

atomic
instruction

test

set

Mutual Exclusion implementation with *test_and_set()*

- Shared data:

boolean lock; //initially lock=FALSE

```
do {
```

```
    while (test_and_set(&lock) )  
        ; // do nothing
```

```
    // critical section
```

```
    lock = FALSE;
```

```
    // remainder section
```

```
} while (TRUE);
```

Process P_i

- Not satisfy bounded-waiting !!
 - It happens some processes may never get the chance to enter CS.

compare_and_swap() Atomic Instruction

- Swap the contents of two words atomically.

atomic instruction

`compare_and_swap(&lock, F0, T1)`

```
int compare_and_swap(int *value, int expected, int new_value)
{
    int temp = *value;
    if (*value == expected)
        *value = new_value;
    return temp;
}
```

test

set

Mutual Exclusion implementation with *compare_and_swap()*

- Shared data:

```
int lock;    // initially lock=0
```

```
while (true) {  
    while (compare and swap(&lock, 0, 1) != 0)  
        ; /* do nothing */  
    /* critical section */  
    lock = 0;  
    /* remainder section */  
}
```

Process P_i

- Not satisfy bounded-waiting !!
 - It happens some processes may never get the chance to enter CS.



Bounded-Waiting Mutual Exclusion with *compare_and_swap()* (satisfies all)

```
boolean waiting[n];  
int lock;
```

Shared variables

```
// initially, waiting[i]=false for all i; lock=0 // false;
```

```
while (true) {
```

```
    waiting[i] = true;
```

```
    key = 1; /* true */
```

```
    while ( waiting[i] && key == 1)
```

```
        key = compare_and_swap(&lock, 0, 1);
```

```
    waiting[i] = false;
```

```
    // critical section
```

```
    j = (i+1) % n;
```

```
    while ( (j != i) && !waiting[j] )
```

```
        j = (j+1) % n;
```

```
    if ( j == i ) then lock = 0; else waiting[j] = false;
```

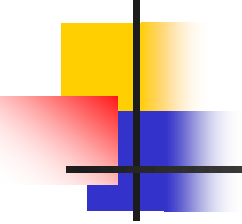
```
    // remainder section
```

Scan order:

0, 1, ..., i-1, i, i+1, i+2, ..., n-1

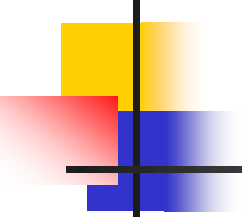
```
}
```

Process P_i



6.4.3 Atomic Variables

- Typically, instructions such as **compare-and-swap** are used as building blocks for constructing other synchronization tools.
- One tool is an **atomic variable** that provides *atomic (uninterruptible)* operations on basic data types such as integers and booleans.
- Most systems that support **atomic variables** provide special atomic data types and functions for accessing and manipulating atomic variables.



Atomic Variables (Cont.)

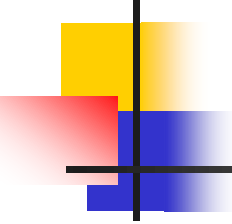
- e.g., the `increment()` operation on the atomic variable `sequence` ensures `sequence` is incremented without interruption:

```
increment(&sequence);
```

- The `increment()` function can be implemented as follows:

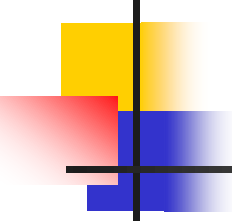
```
void increment(atomic_int *v)
{
    int temp;
    do {
        temp = *v;
    }
    while (temp != (compare_and_swap(v,temp,temp+1)));
}
```





6.5 Mutex Locks

- Hardware-based solutions to critical-section problem (using **test_and_set()** and **compare_and_swap()**) are complicated and generally inaccessible to application programmers.
難以到達的; 難以使用的
- OS designers build higher-level software tools to solve critical section problem



Mutex Locks (Cont.)

- The simplest is **mutex lock** (*mutual exclusion*).
 - Use mutex lock to protect critical regions and thus prevent race conditions.
 - A process must **acquire the lock** before entering a critical section and **release the lock** when it exits the critical section.
- **Calls to `acquire()` and `release()` must be performed atomically**
 - Usually implemented via H/W atomic instructions

Solution to the critical-section problem using mutex locks

indicating if lock is available or not

```
while (true) {  
    acquire lock  
    critical section  
    release lock  
    remainder section  
}
```

```
boolean available;  
    //initially true  
acquire() {  
    while (!available)  
        ; /* busy wait */  
    available = false;  
}  
release() {  
    available = true;  
}
```

Must be performed atomically

Mutex Locks (Cont.)



- ***busy waiting***

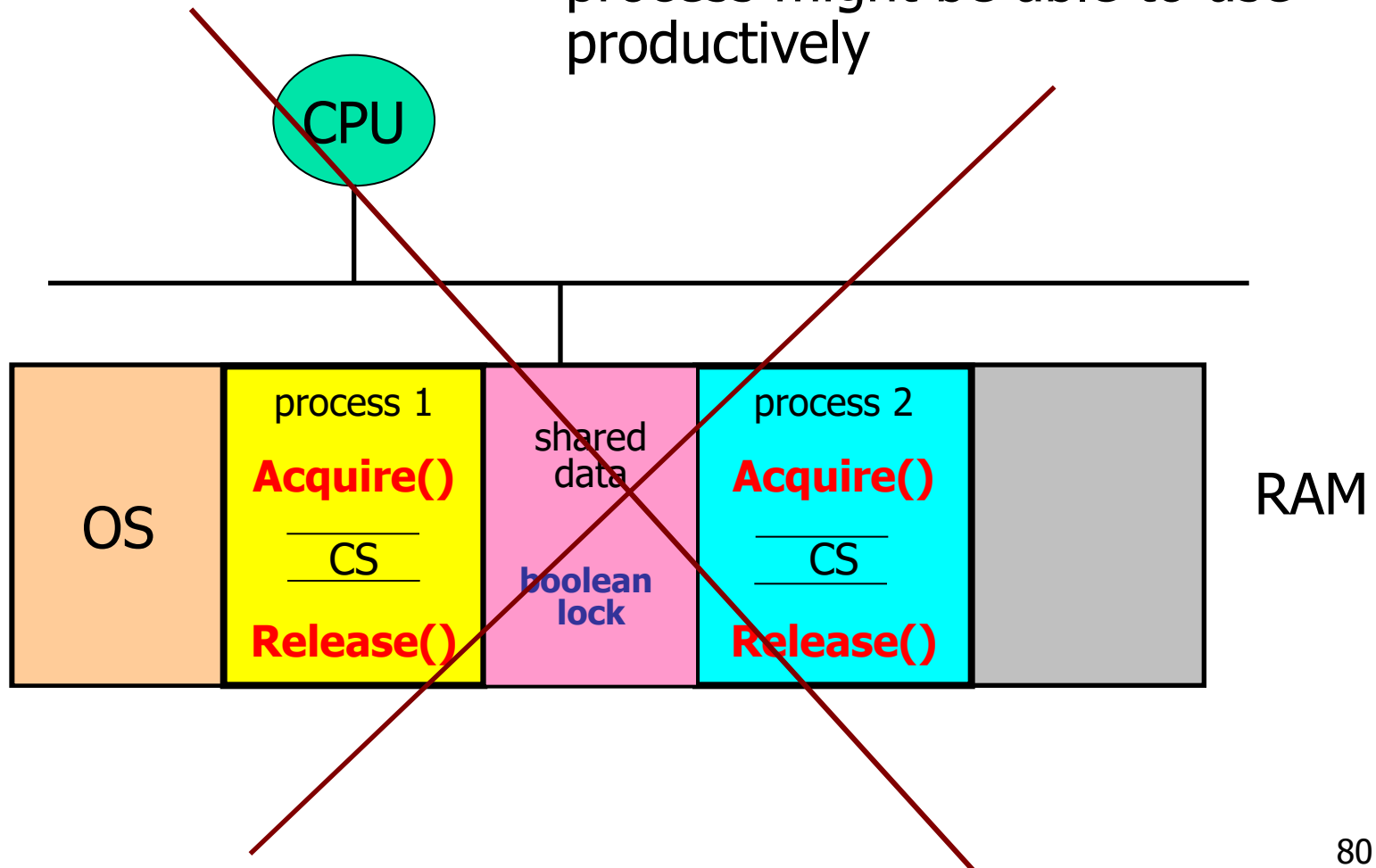
- when a process is in its critical section, any other process that tries to enter its critical section must loop continuously in the call to acquire()
- Problem in a real multiprogramming system, where a single CPU core is shared among many processes
- wastes CPU cycles that some other process might be able to use productively
- This type of mutex lock is also called a **spinlock**.

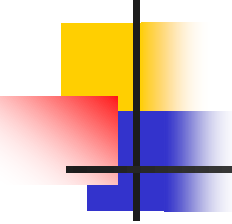
spin 旋轉；自旋



Uniprocessor (using spinlock busy waiting)

wastes CPU cycles that some other process might be able to use productively

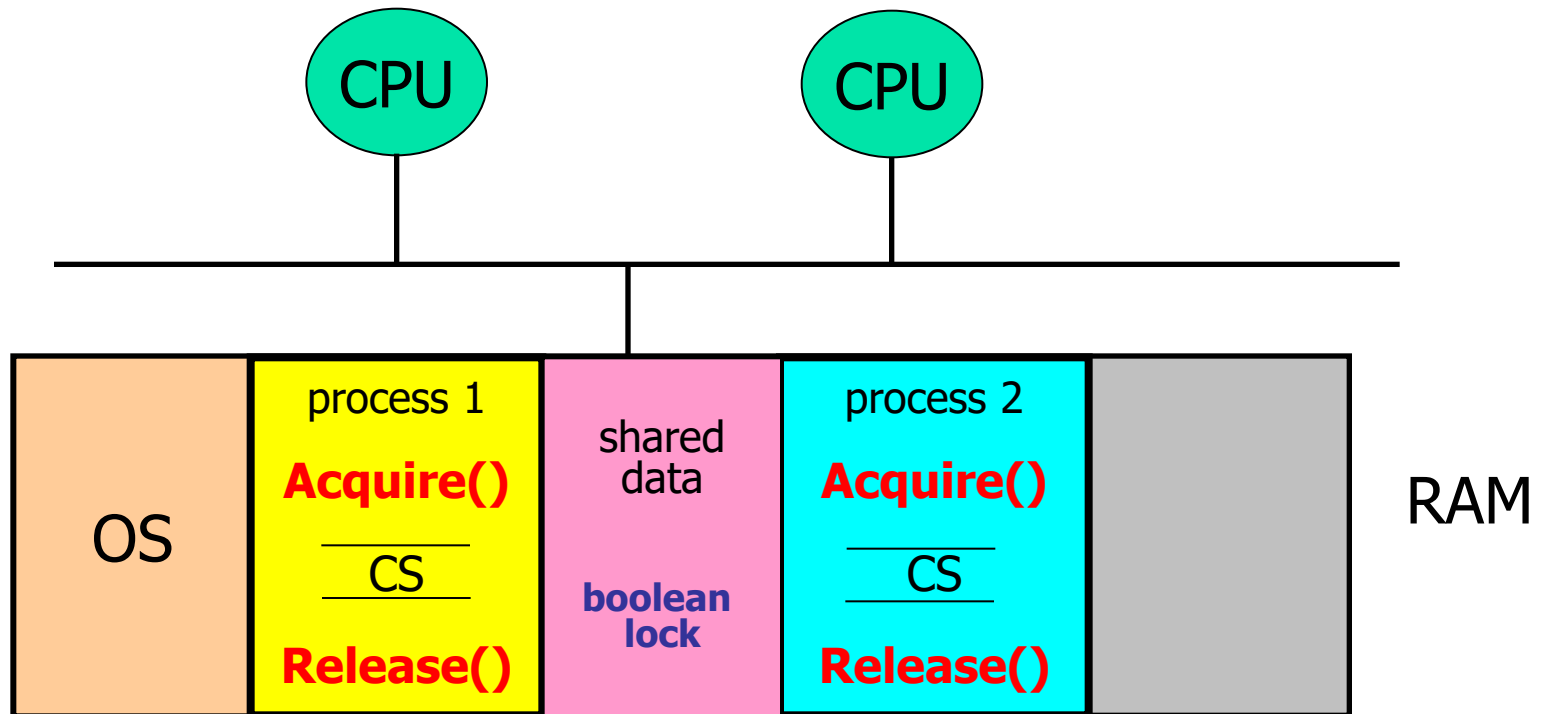


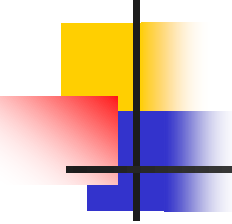


Mutex Locks (Cont.)

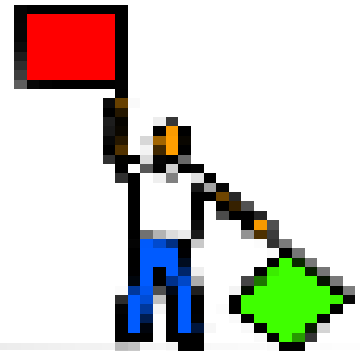
- In multicore systems, **spinlocks** are useful when locks are expected to be held for a short duration
 - Where one thread can “spin” on one processor while another thread performs its critical section on another processor.
 - no context switch is required when a process must wait on a lock, and a context switch may take considerable time

Multiprocessor (using spinlock for short CS)





6.6 Semaphores



- **Mutex locks** are generally considered the simplest of **synchronization tool**.
- A more robust **synchronization tool** that can provide more sophisticated ways for processes to synchronize their activities: ***semaphore***.

Semaphores (Cont.)

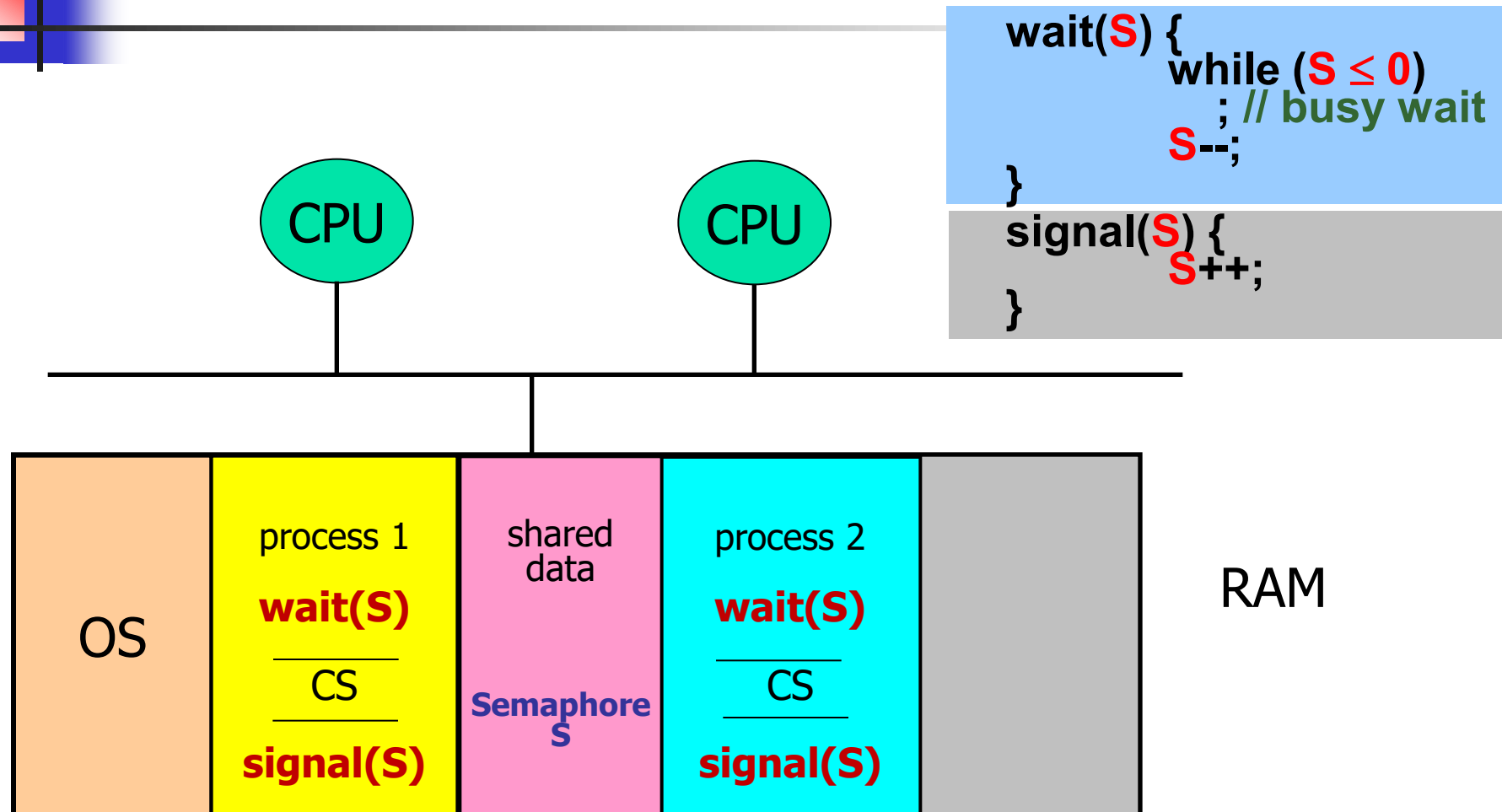
- A semaphore **S** is an integer variable
- Can be accessed only through two standard *atomic* operations
 - *wait* (**P** operation)
 - *signal* (**V** operation)

```
wait(S) {  
    while (S ≤ 0)  
        ; // busy wait  
    S--;  
}  
  
signal(S) {  
    S++;  
}
```

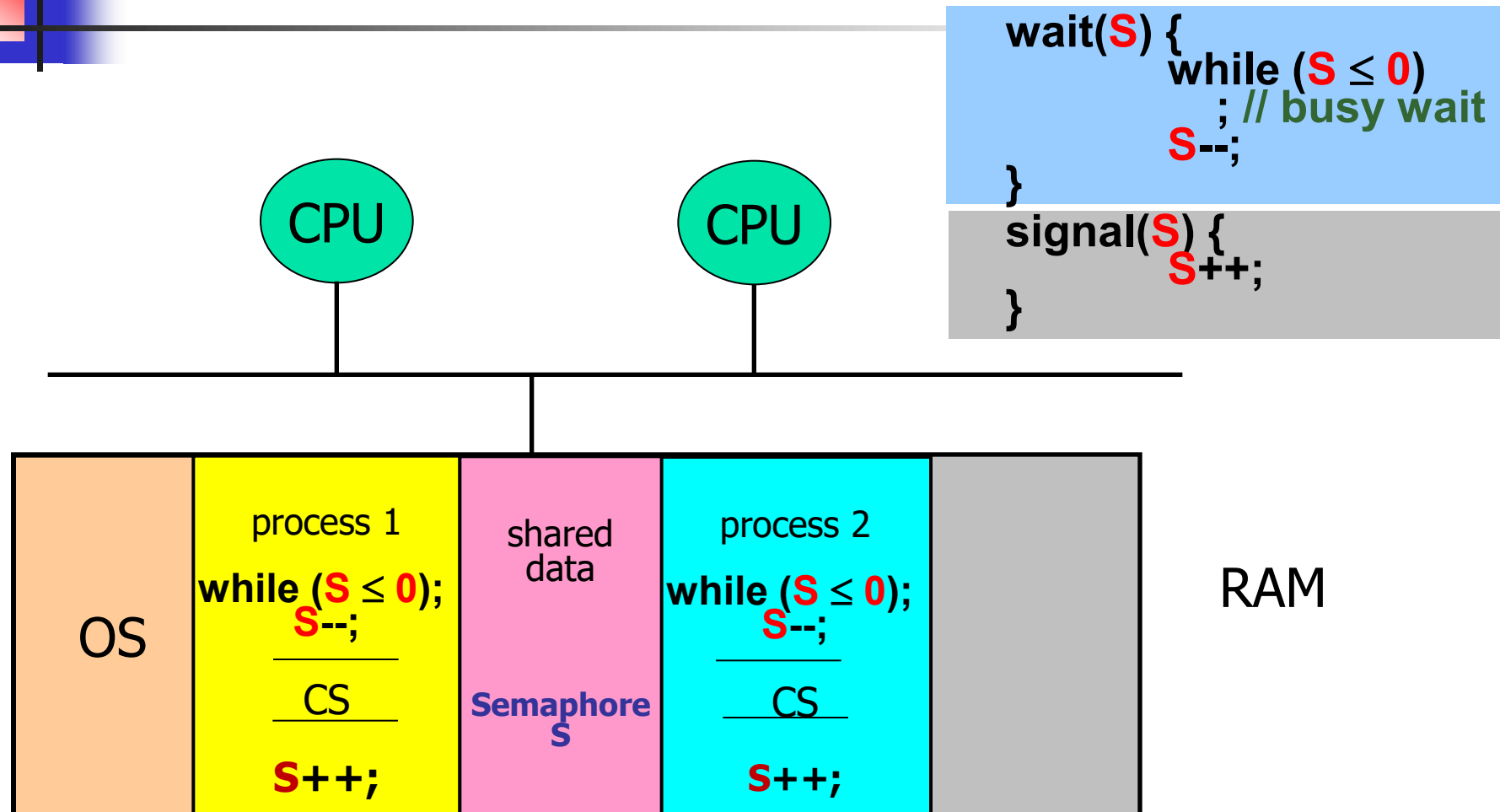
From the Dutch:
P for wait to test
V for signal to increment



Using semaphore for mutual exclusion



Using semaphore for mutual exclusion



6.6.1 Semaphore Usage

```
wait(S) {  
    while (S ≤ 0)  
        ; // busy wait  
    S--;  
}  
signal(S) {  
    S++;  
}
```

- Two types of semaphores:
 - (1) **Binary semaphore**
 - a semaphore with an integer value that can range only between 0 and 1
 - Behave similarly to mutex, as they provide **mutual exclusion**.
 - Can be used to deal with critical-section problem for multiple processes
 - The semaphore is initialized to 1

Semaphore Usage (Cont.)

```
wait(S) {  
    while (S ≤ 0)  
        ; // busy wait  
    S--;  
}  
signal(S) {  
    S++;  
}
```

- Two types of semaphores: (Cont.)
 - (2) *Counting semaphore*
 - its integer value can range over an unrestricted domain
 - Can be used to control access to a given resource consisting of a finite number of instances.
 - The semaphore is initialized to

the number of resources available

Semaphore Usage

-(1) mutual exclusion

```
wait(S) {  
    while (S ≤ 0)  
        ; // busy wait  
    S--;  
}  
  
signal(S) {  
    S++;  
}
```

- Example: critical-section problem for n -processes

- Shared data:

semaphore mutex; // initially **mutex** = 1

```
do {  
    wait (mutex);  
    // critical section  
    signal (mutex);  
    // remainder section  
} while (TRUE);
```

Process P_i

Semaphore Usage

- (2) synchronization

```
wait(S) {  
    while (S ≤ 0)  
        ; // busy wait  
    S--;  
}  
  
signal(S) {  
    S++;  
}
```

- Semaphore as general synchronization tool to synchronize processes
 - Use semaphore (with value initialized to **0**)
- Example: if we want to execute S1 and then S2

- Shared data:

semaphore synch; // initially *synch* = 0

S1;
signal(synch);

Process P_1

wait(synch);
S2;

Process P_2

6.6.2 Semaphore Implementation

- *busy waiting*

- The implementation of *mutex locks*

```
boolean available; //initially true
```

```
acquire() {  
    while (!available)  
        ; /* busy wait */  
    available = false;  
}
```

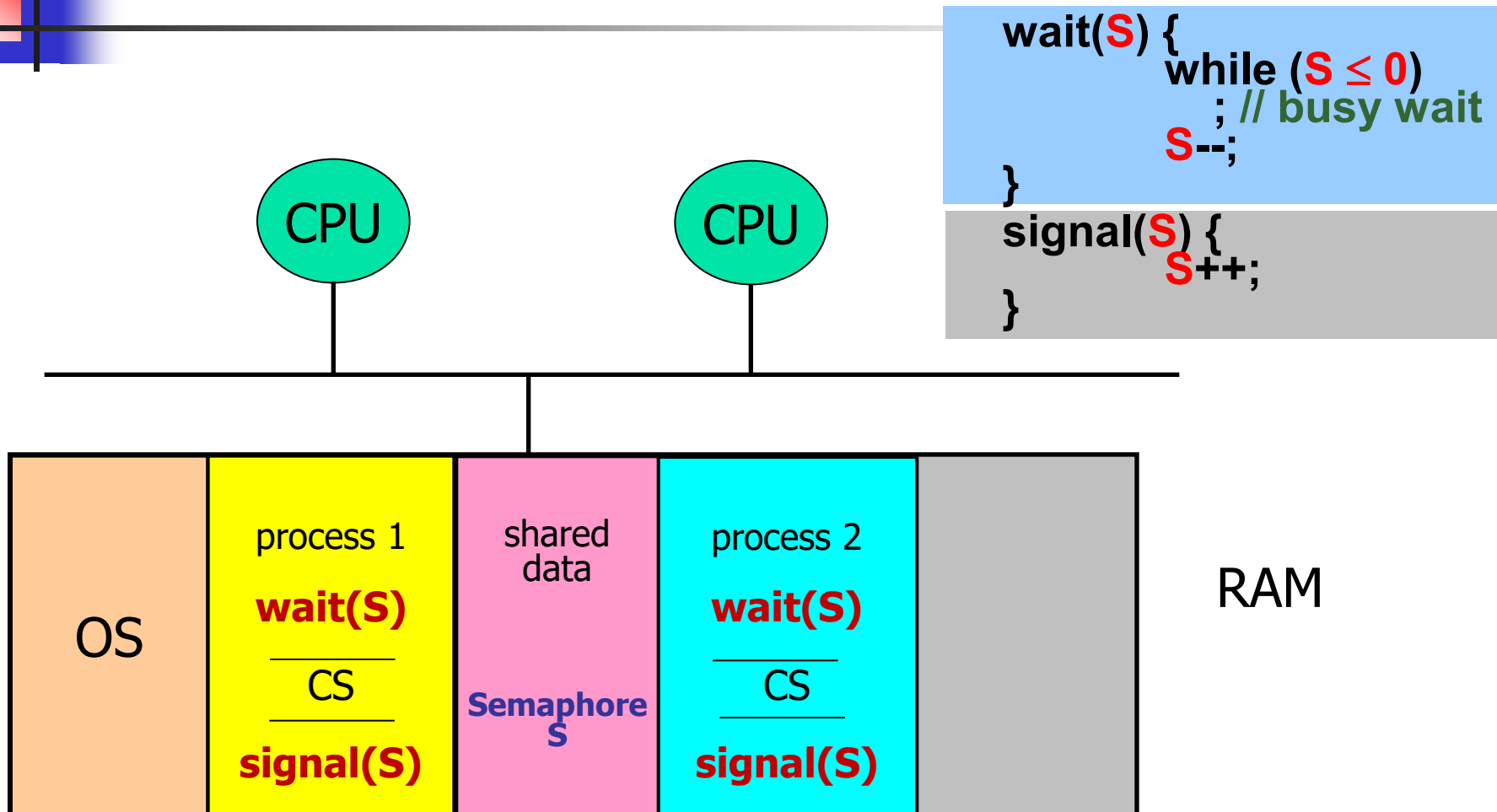
```
release() {  
    available = true;  
}
```

- The implementation of *semaphore*

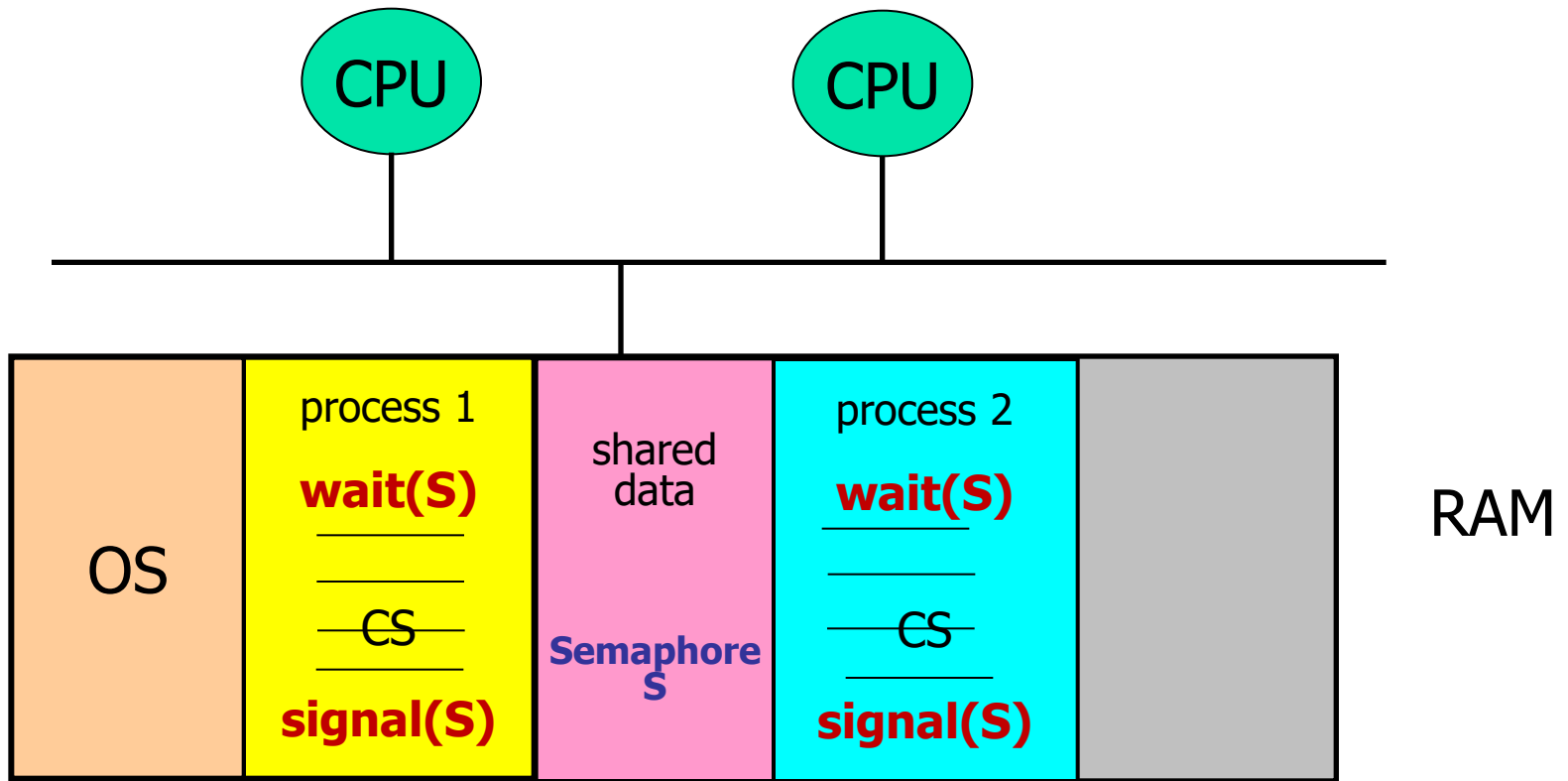
```
wait(S) {  
    while (S ≤ 0)  
        ; // busy wait  
    S--;  
}
```

```
signal(S) {  
    S++;  
}
```

Multiprocessor (using busy waiting for short CS)

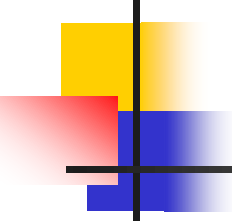


Multiprocessor (using **blocking** semaphore for long CS)



Semaphore Implementation - Without Busy Waiting (Blocking Semaphores)

- To overcome the busy waiting, we can use the concept of *sleep* and *wakeup* operations.
- When a process has to **wait** on a semaphore
 - place the process into a waiting queue associated with the semaphore
 - switch the state of the process to the **waiting** state
 - transfer the control to the CPU scheduler
 - select another process to execute
- When a process executes **signal** operation
 - **wakeup** a process by changing its state to **ready**
 - place the process in the ready queue



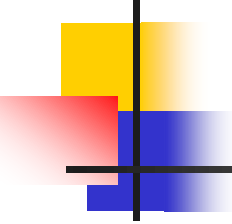
Implementation of Blocking Semaphores

- Define a semaphore as a “C” struct:

```
typedef struct {  
    int value;  
    struct process *list;  
} semaphore;
```

each semaphore has an associated waiting queue

- Assume two simple operations:
 - **sleep()**: suspends the process that invokes it
 - **wakeup(*P*)**: resume the execution of a suspended process *P*
 - provided by OS as basic system calls



Implementation of Blocking Semaphores (Cont.)

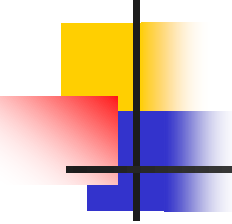
- Semaphore operations now defined as:

wait(S)

```
void wait(semaphore *S) {  
    S->value --;  
    if (S->value < 0) {  
        add this process to S->list;  
        sleep(); // wait on waitQ, context switch  
    }  
}
```

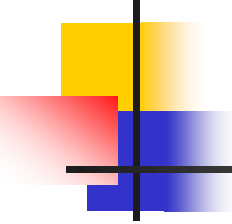
signal(S)

```
void signal(semaphore *S) {  
    S->value ++;  
    if (S->value <= 0) {  
        remove a process P from S->list;  
        wakeup(P); // from waitQ to readyQ  
    }  
}
```



Implementation of Blocking Semaphores (Cont.)

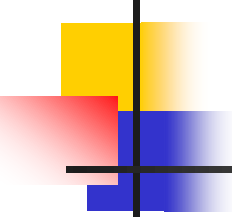
- If the semaphore value is negative, its magnitude is the number of processes waiting on that semaphore.
- To ensure bounded waiting, waiting queue is maintained as a FIFO queue



Atomicity of *Signal* and *Wait* Operations

- **signal() and wait() should be executed atomically**
- **No two processes can execute wait() and signal() operations on the same semaphore at the same time.**
- **This is a critical-section problem where the wait and signal code are placed in the critical section**

量;大小



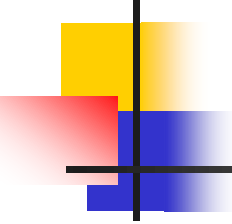
Signal and wait should be executed atomically

- **In a single-processor environment**

- **disable interrupt** when **wait()** and **signal()** operations are executing
 - Once interrupts are inhibited, instructions from different processes cannot be interleaved. Only the currently running process executes until interrupts are reenabled and the scheduler can regain control.

- **In a multicore environment**

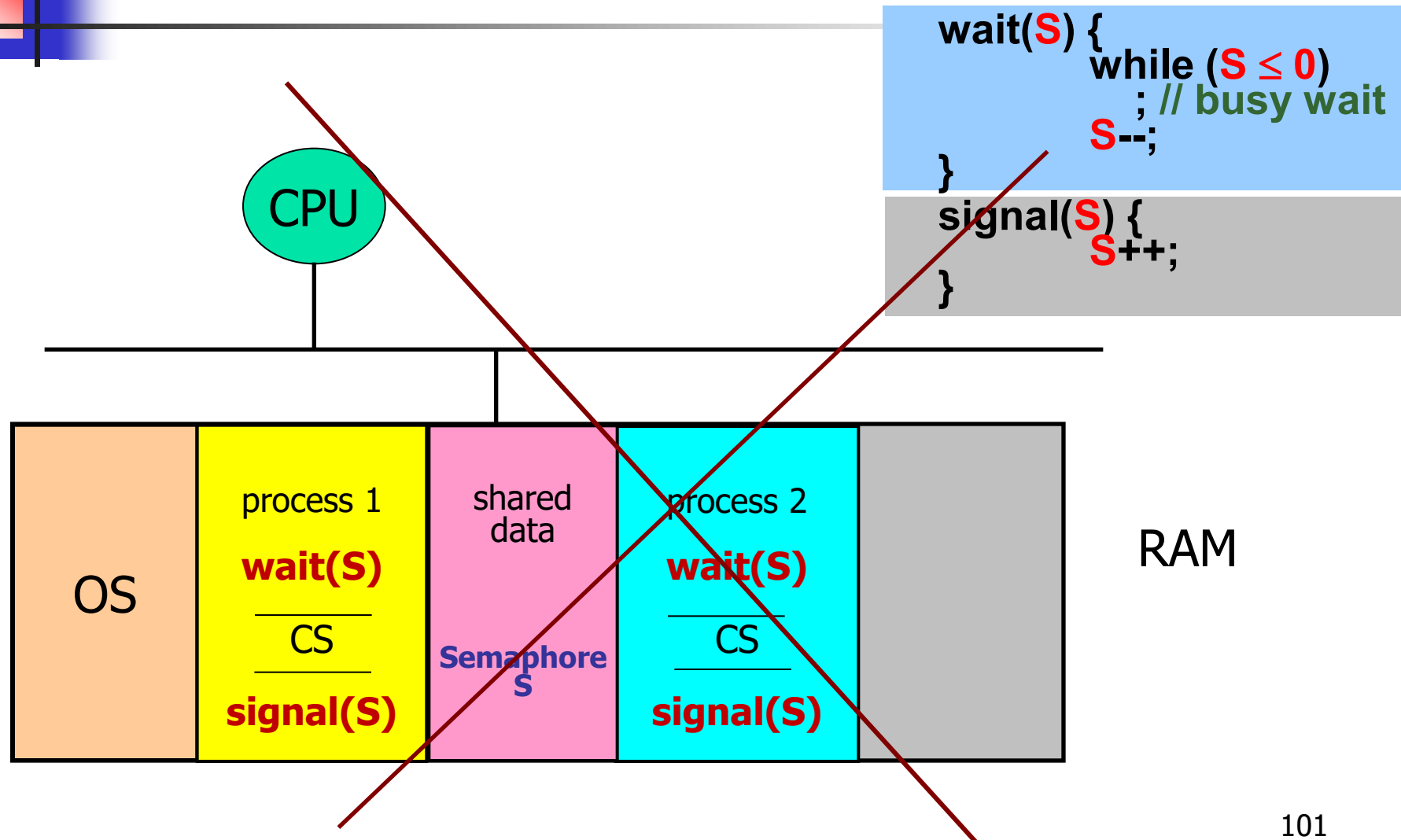
- ~~Interrupts must be disabled on every processing core; otherwise, instructions from different processes (running on different cores) may be interleaved in some arbitrary way.~~
 - ~~Disabling interrupts on every core can be a difficult task and can seriously diminish performance.~~
- Therefore, **SMP** systems must provide alternative **locking** techniques – such as **compare_and_swap()** or **spinlocks** – to ensure that **wait()** and **signal()** are performed atomically.



Why **spinlocks** are not appropriate for single-processor systems yet are often used in multiprocessor systems?

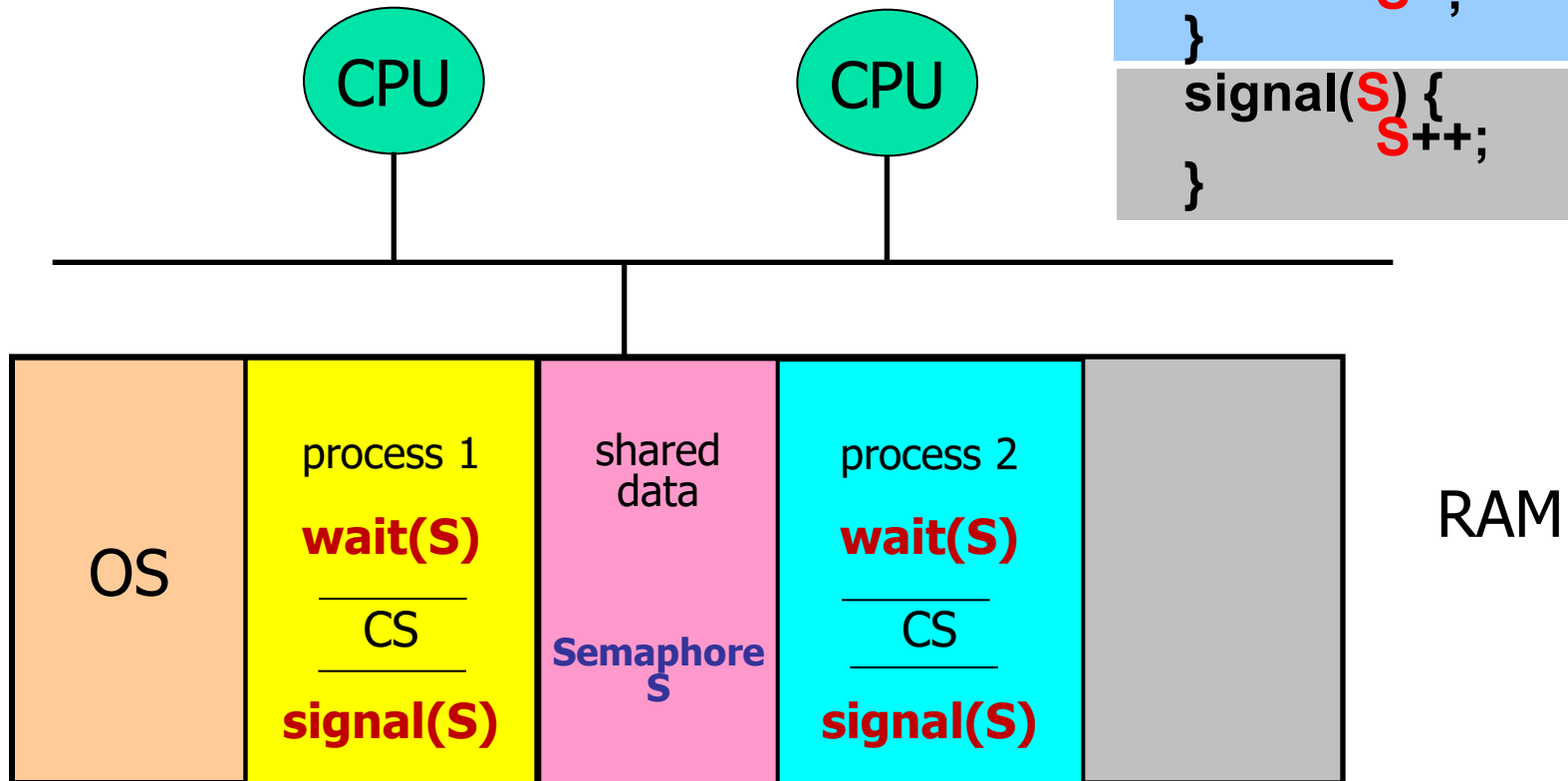
- Because the condition that would break a process out of the spinlock can be obtained only by executing a different process.
- In a **uniprocessor** system
 - If the process is not relinquishing the processor, other processes do not get the opportunity to set the program condition required for the first process to make progress.
- In a **multiprocessor** system
 - other processes execute on other processors and thereby modify the program state in order to release the first process from the spinlock

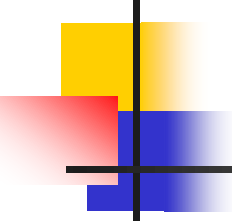
Uniprocessor (using spinlock busy waiting)



Multiprocessor (using spinlock busy waiting)

```
wait(S) {  
    while (S ≤ 0)  
        ; // busy wait  
    S--;  
}  
signal(S) {  
    S++;  
}
```

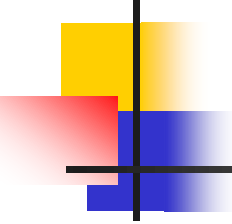




6.7 Monitors

- Semaphores provide a convenient and effective mechanism for process synchronization.
- However, **incorrect use of semaphores can result in timing errors.** 
 - Caused by an honest programming error or an uncooperative programmer.
- **Introduce high-level language constructs**
 - **Monitor** 





Incorrect Use of Semaphores

- Incorrect use of semaphores can result in timing errors

```
signal(mutex);  
...  
critical section  
...  
wait(mutex);
```

violate
mutual exclusion

```
wait(mutex);  
...  
critical section  
...  
wait(mutex);
```

permanently
block

```
wait(mutex);  
...  
critical section  
...  
[gray bar]
```

```
[gray bar]  
...  
critical section  
..  
signal(mutex);
```

violate mutual exclusion or
cause deadlock?



6.7.1 Monitor Usage

Syntax of a monitor

```
monitor monitor-name
{
    //shared variable declarations

    function P1(...) {
        ...
    }

    function P2(...) {
        ...
    }

    ...

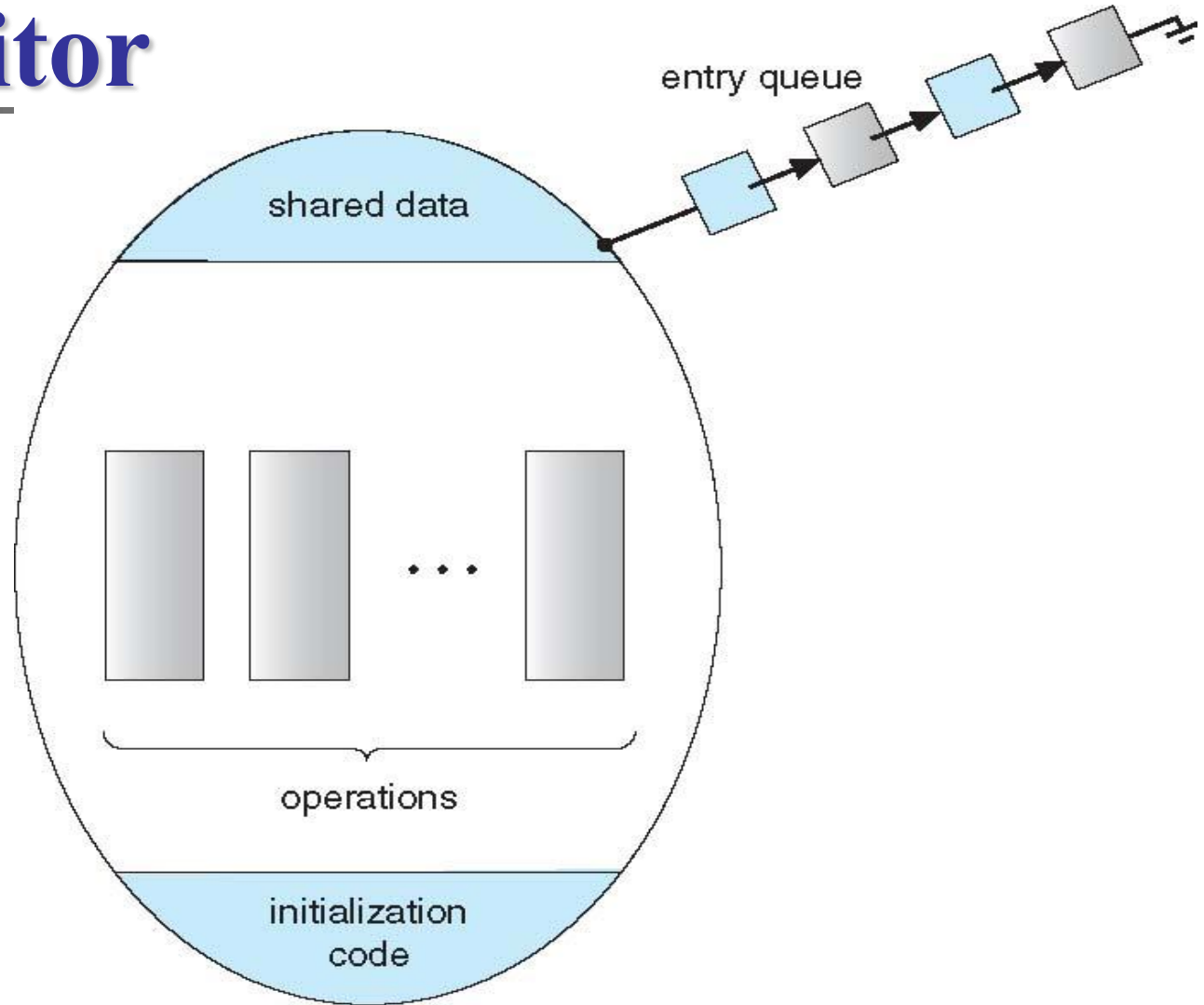
    function Pn(...) {
        ...
    }

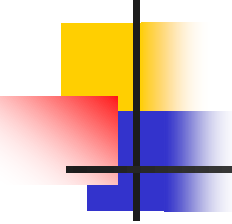
    initialization_code (...) {
        ...
    }
}
```

- **high-level synchronization construct**
 - provides a convenient and effective mechanism for process synchronization
- A ***monitor*** type is an abstract data type that includes a set of **programmer-defined operations** that are provided with mutual exclusion within the monitor.

圖式的

Schematic View of a Monitor

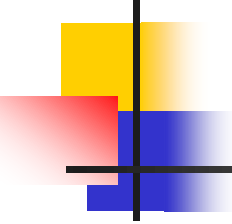




Monitors (Cont.)

- The monitor type declares
 - **the variables** whose values define the state of an instance of that type
 - **the bodies of functions** that operate on those variables.
- The local variables of a monitor can be accessed by only the local functions
- The monitor construct ensures that **only one process at a time can be active within the monitor.**
 - The programmer does not need to code this synchronization constraint explicitly.
- But not powerful enough to model some synchronization schemes





Condition Construct

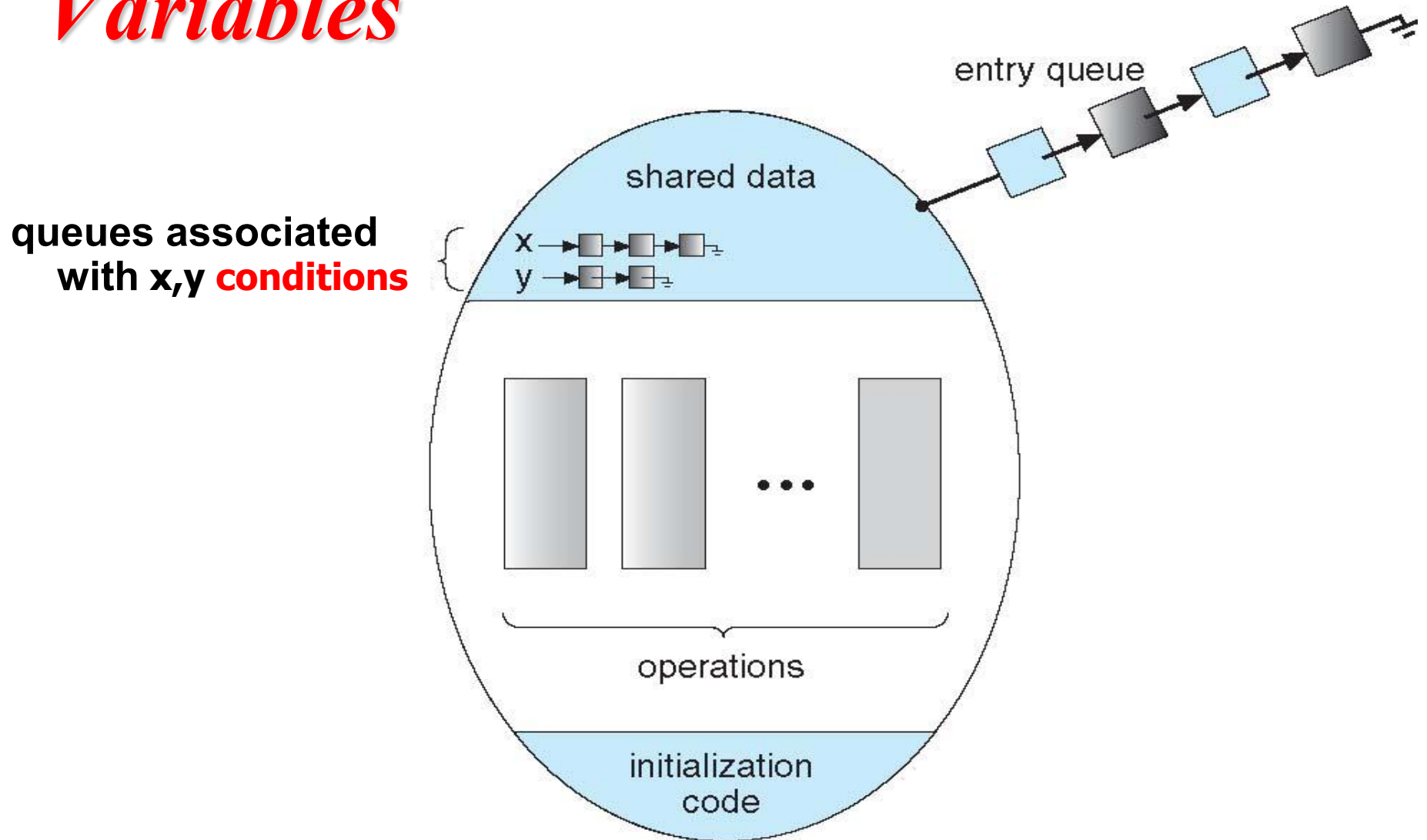
- To allow a process to wait within the monitor, a **condition variable** must be declared, as:

condition ***x, y;***

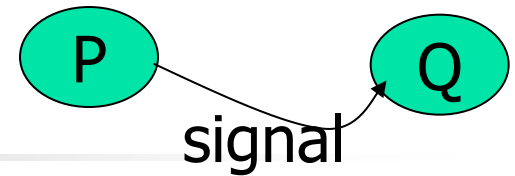
特製的

- A programmer who needs to write a tailor-made synchronization scheme can define one or more variables of type ***condition***
- **Two operations on a condition variable:**
 - ***x.wait()***: process invoking this operation is suspended until another process invokes ***x.signal()***
 - ***x.signal()***: resumes exactly one suspended process.
 - If no process is suspended, the ***signal()*** operation has no effect.

Monitor with *Condition Variables*



Condition Variables Choices



- If process P invokes **x.signal ()**, with Q in **x.wait ()** state, what should happen next?
 - If Q is resumed, then P must wait
- Options include
 - **Signal and wait** – P waits until Q leaves monitor or waits for another condition
 - **Signal and continue** – Q waits until P leaves the monitor or waits for another condition
 - Both have pros and cons – language implementer can decide
 - Monitors implemented in Concurrent Pascal compromise
 - P executing signal immediately leaves the monitor, Q is resumed
 - Implemented in other languages including C#, Java, Erlang

6.7.2 Monitor Implementation Using Semaphores

- Variables

```
semaphore mutex; // (initially = 1)
semaphore next;  // (initially = 0)
int next_count = 0;
```

- Each procedure F will be replaced by

```
wait(mutex);
```

```
...
```

```
body of  $F$ ;
```

```
...
```

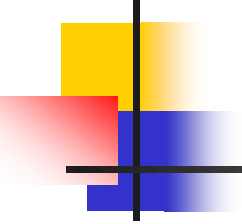
```
if (next_count > 0)
```

```
    signal(next);
```

```
else
```

```
    signal(mutex);
```

- Mutual exclusion within a monitor is ensured



Monitor Implementation – Condition Variables

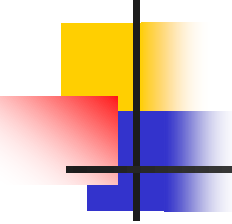
- For each condition variable x , we have:

```
semaphore x_sem; // (initially = 0)
int x_count = 0;
```

- The operation **$x.\text{wait}()$** :
- The operation **$x.\text{signal}()$** :

```
x_count++;
if (next_count > 0)
    signal(next);
else
    signal(mutex);
wait(x_sem);
x_count--;
```

```
if (x_count > 0) {
    next_count++;
    signal(x_sem);
    wait(next);
    next_count--;
}
```

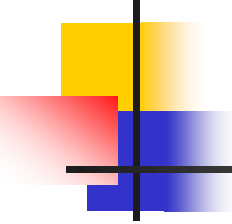


6.7.3 Resuming Processes Within a Monitor

- **Process-resumption order within a monitor**
- ***Conditional-wait* construct**

$x.wait(c)$

- c – integer expression evaluated when the wait() operation is executed.
- value of c is *priority number*
- when $x.signal()$ is executed, process with smallest priority number (highest priority) is resumed next.



Example: the use of Conditional-Wait Construct

- The ResourceAllocator monitor (shown in Fig. 6.14) controls the allocation of a single resource among competing processes.
 - A process that needs to access the resource in question must observe the sequence:

R.acquire(t); ← Max time it plans to use the resource

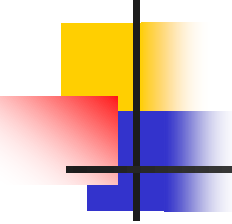
...

access the resource;

...

R.release(); ← The released resource will be allocated to the waiting process with the shortest time-allocation request.

Where *R* is an instance of type ResourceAllocator

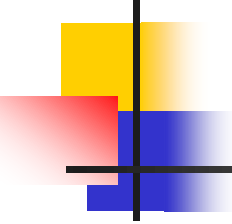


A monitor to allocate a single resource (Fig. 6.14)

```
monitor ResourceAllocator
{
    boolean busy;
    condition x;

    void acquire(int time) {
        if (busy)
            x.wait(time);
        busy=TRUE;
    }

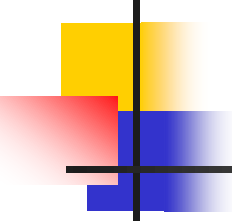
    void release() {
        busy=FALSE;
        x.signal();
    }
    initialization_code() {
        busy=FALSE;
    }
}
```



Should Correctly Use the Monitor

- **Check two conditions to establish correctness of system:**
 - **User processes must always make their calls on the monitor in a correct sequence.**
 - **Must ensure that an uncooperative process does not ignore the mutual-exclusion gateway provided by the monitor, and try to access the shared resource directly, without using the access protocols.**





6.8 Liveness

- Processes may have to wait indefinitely while trying to acquire a synchronization tool such as a mutex lock or semaphore.
- Waiting indefinitely violates the progress and bounded-waiting criteria discussed at the beginning of this chapter.
- **Liveness** refers to a set of ^{特性}properties that a system must satisfy to ensure processes make progress during their execution life cycle.
- Indefinite waiting is an example of a liveness failure.

活性,
活躍性

6.8.1 Deadlock

```
wait(S) {  
    while (S ≤ 0)  
        ; // busy wait  
    S--;  
}  
signal(S) {  
    S++;  
}
```

- **deadlock**: two or more processes are waiting indefinitely for an event that can be caused **only** by one of the waiting processes.
- Let S and Q be two semaphores, initialized to 1.

P0

```
wait(S);  
wait(Q);  
⋮  
signal(S);  
signal(Q);
```

P1

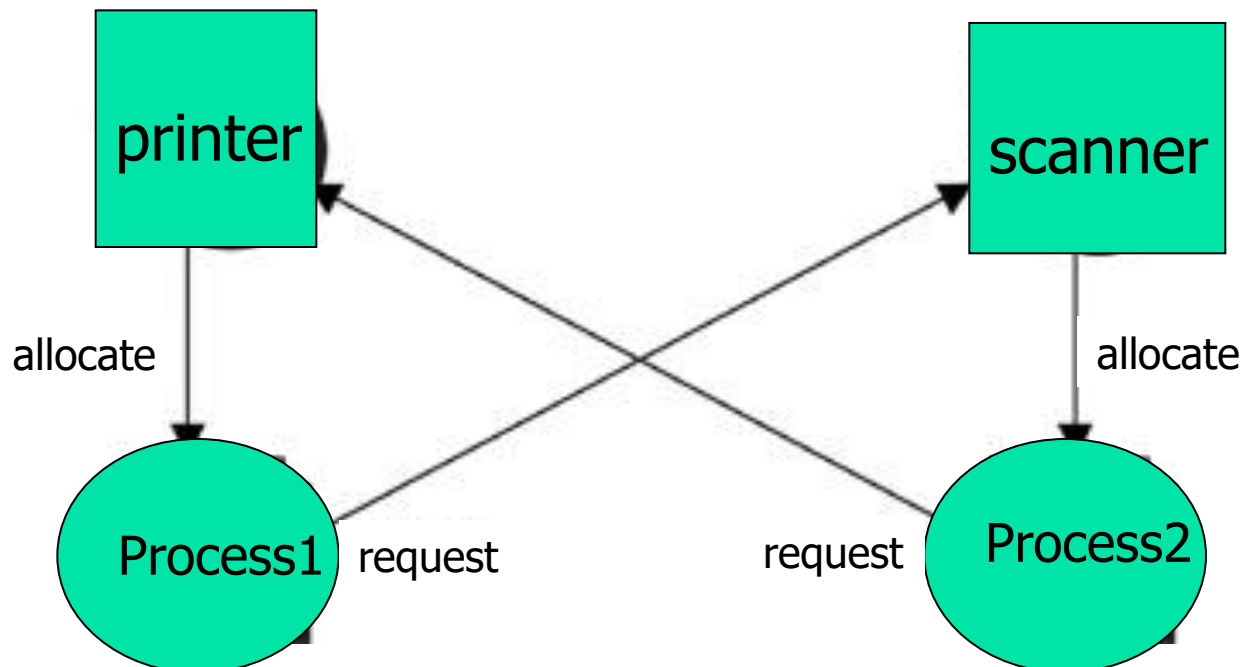
```
wait(Q);  
wait(S);  
⋮  
signal(Q);  
signal(S);
```

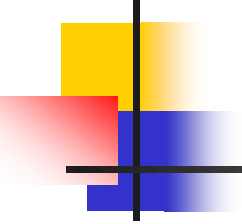
P0: wait(S) ... pass
P1: wait(Q) ... pass
P0: wait(Q) ... block
P1: wait(S) ... block

P₀ and P₁ are
deadlocked !

Deadlock situation between two tasks

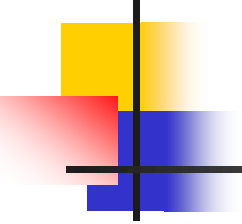
- Situation: multiple concurrent threads of execution in a system are blocked permanently because of resource requirements that can never be satisfied.





Deadlock (Cont.)

- A set of processes is in a deadlocked state
 - when every process in the set is waiting for an event that can be caused only by another process in the set.



6.8.2 Priority Inversion

反轉; 倒轉

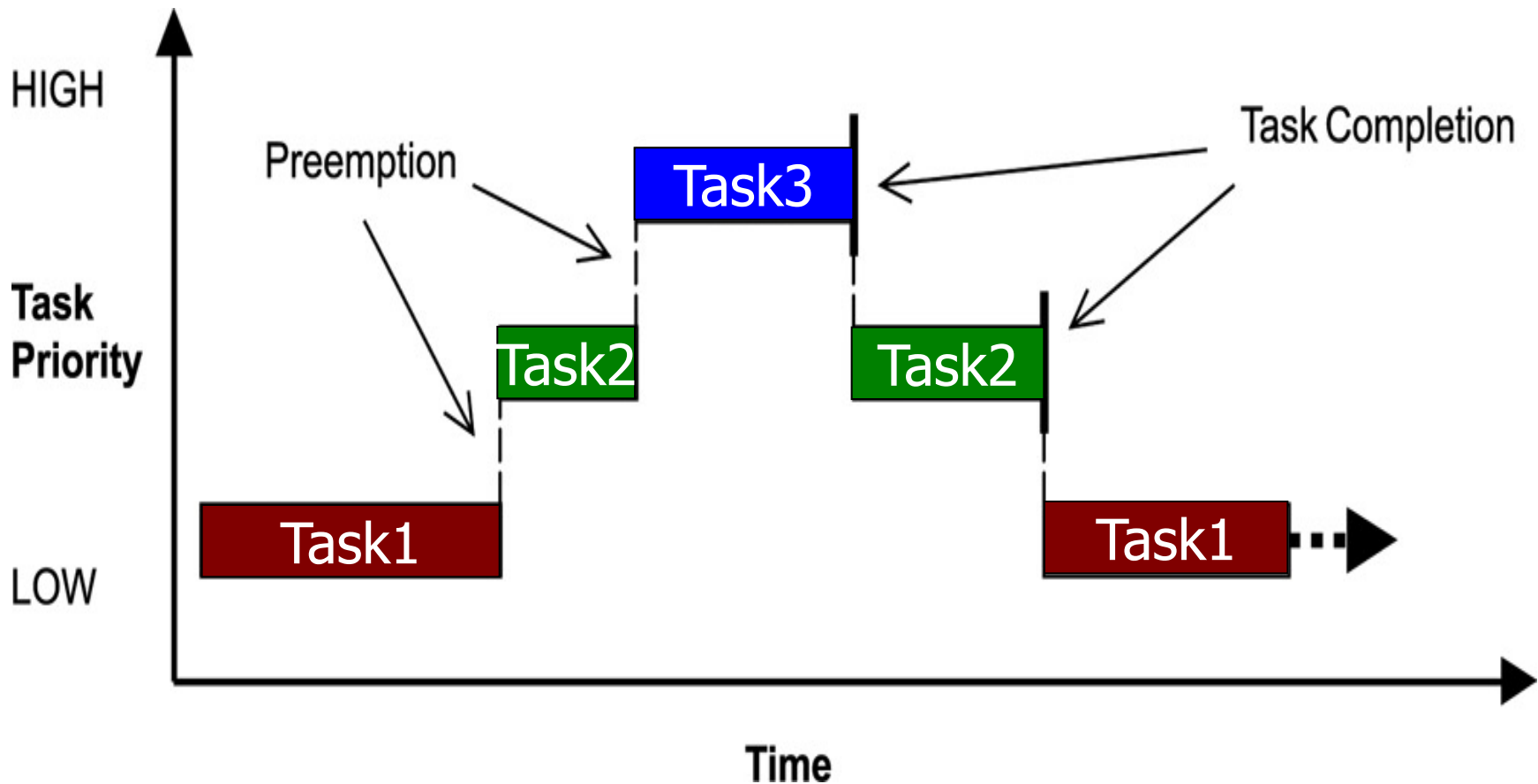
■ Priority Inversion

- Scheduling problem when lower-priority process holds a lock needed by higher-priority process
- eg., Processes with $L < M < H$ priorities, process H requires resource currently being accessed by process L. Since process M may preempt process L, M affects how long H must wait for L to relinquish resource

■ Priority-inheritance protocol

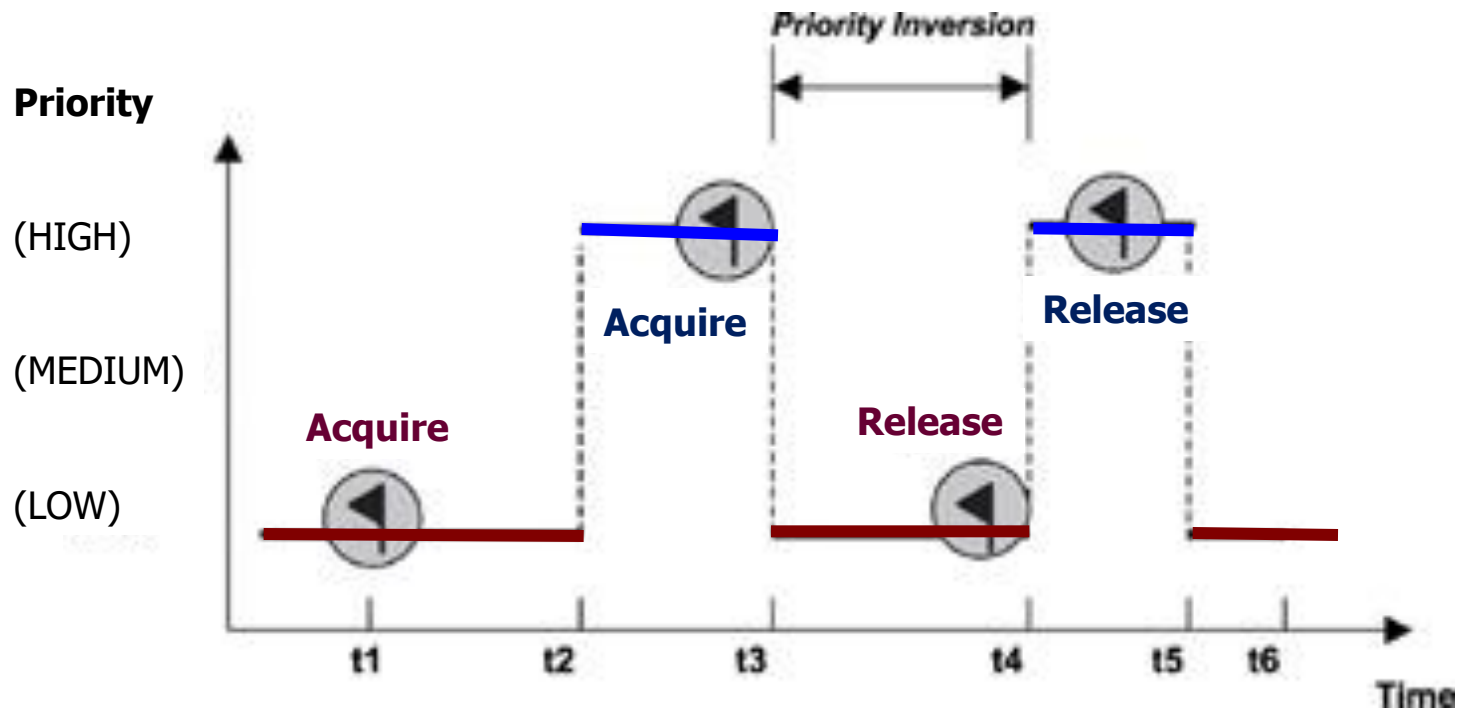
- Temporarily let the lower-priority process inherit the higher priority, thereby preventing medium-priority processes from preempting its execution

Preemptive Priority-Based Scheduling

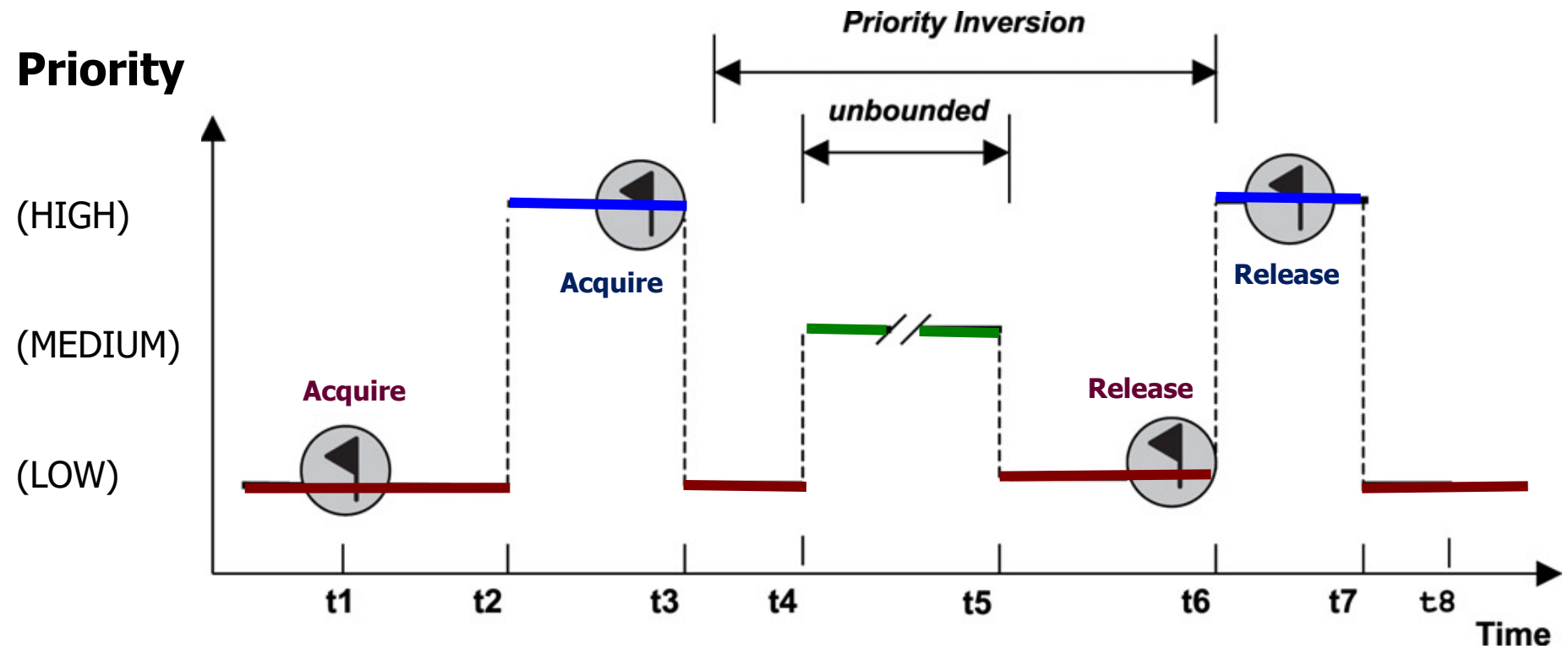


Priority Inversion Example

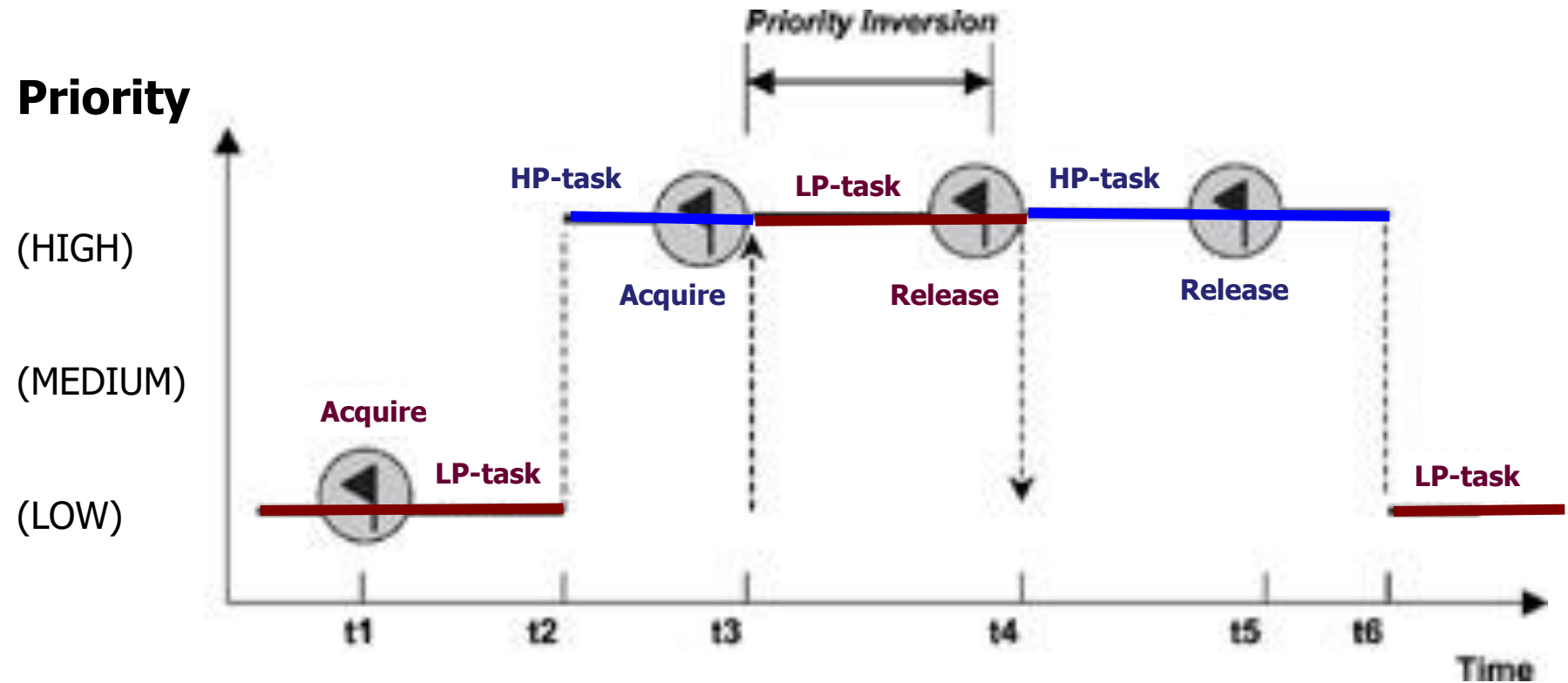
- A situation in which a low-priority task executes while a higher priority task waits on it due to **resource contentions**.

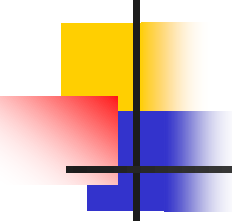


Unbounded Priority Inversion Example



Priority Inheritance Protocol Example





Homework (2擇1)

- 手寫作業: (100%)

- (75%) Ex. 6.3, 6.4, 6.6, 6.15, 6.17, 6.24

- HW (25 %)

- 1.至Youtube中尋找與 Ch6&7 課程相關的影片。
 - 2.附上影片 URL。
 - 3.請說明影片內容。

- 製作影片 (100%)

- 1.至多2人一組。
 - 2.上傳至 moodle。
 - 3.請說明影片內容。

